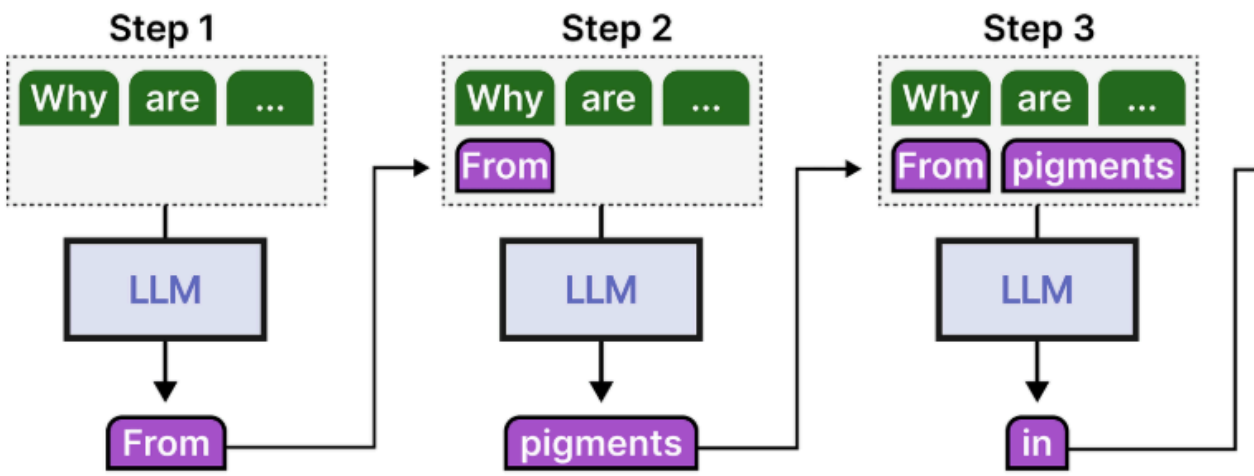
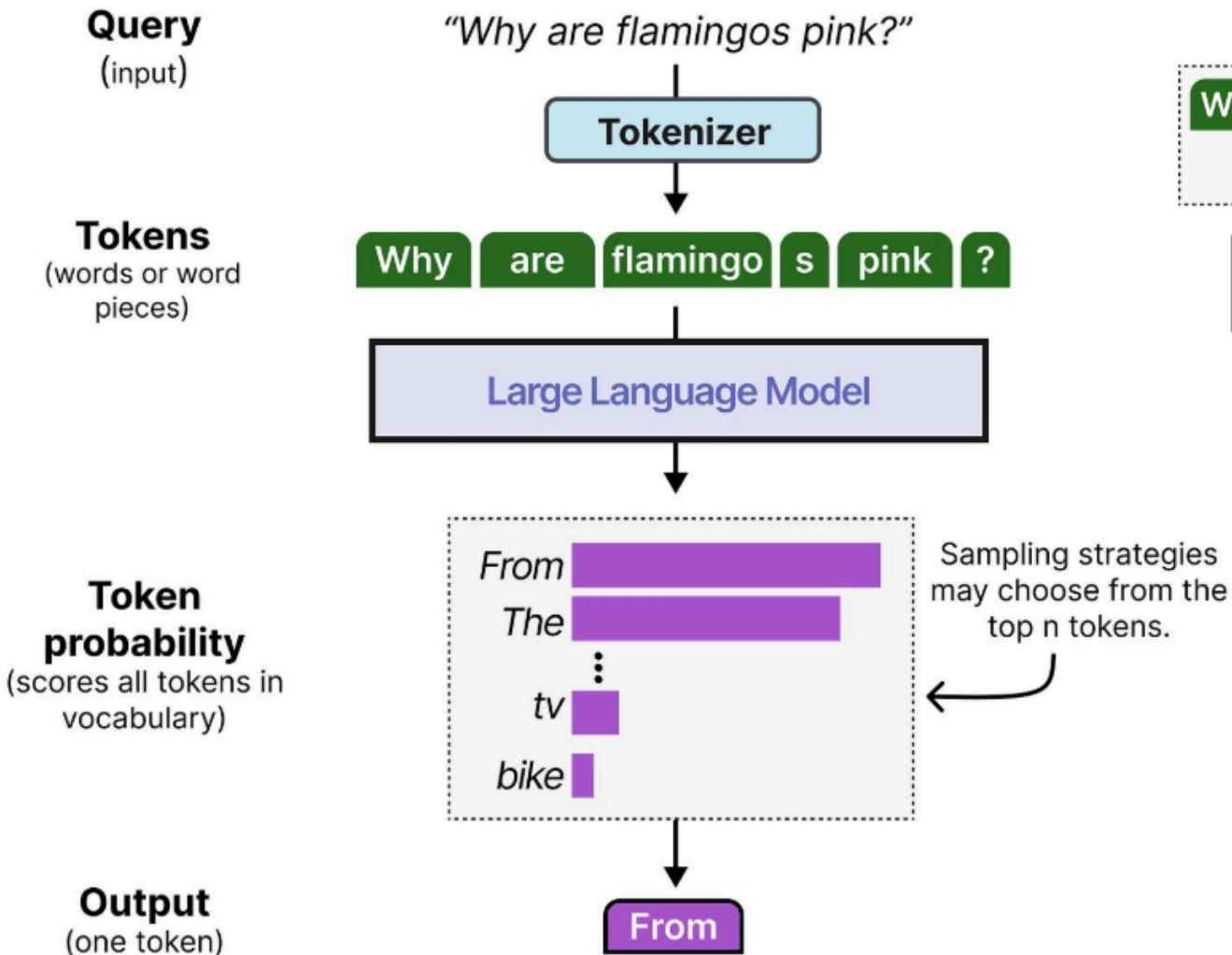


More on LLMs

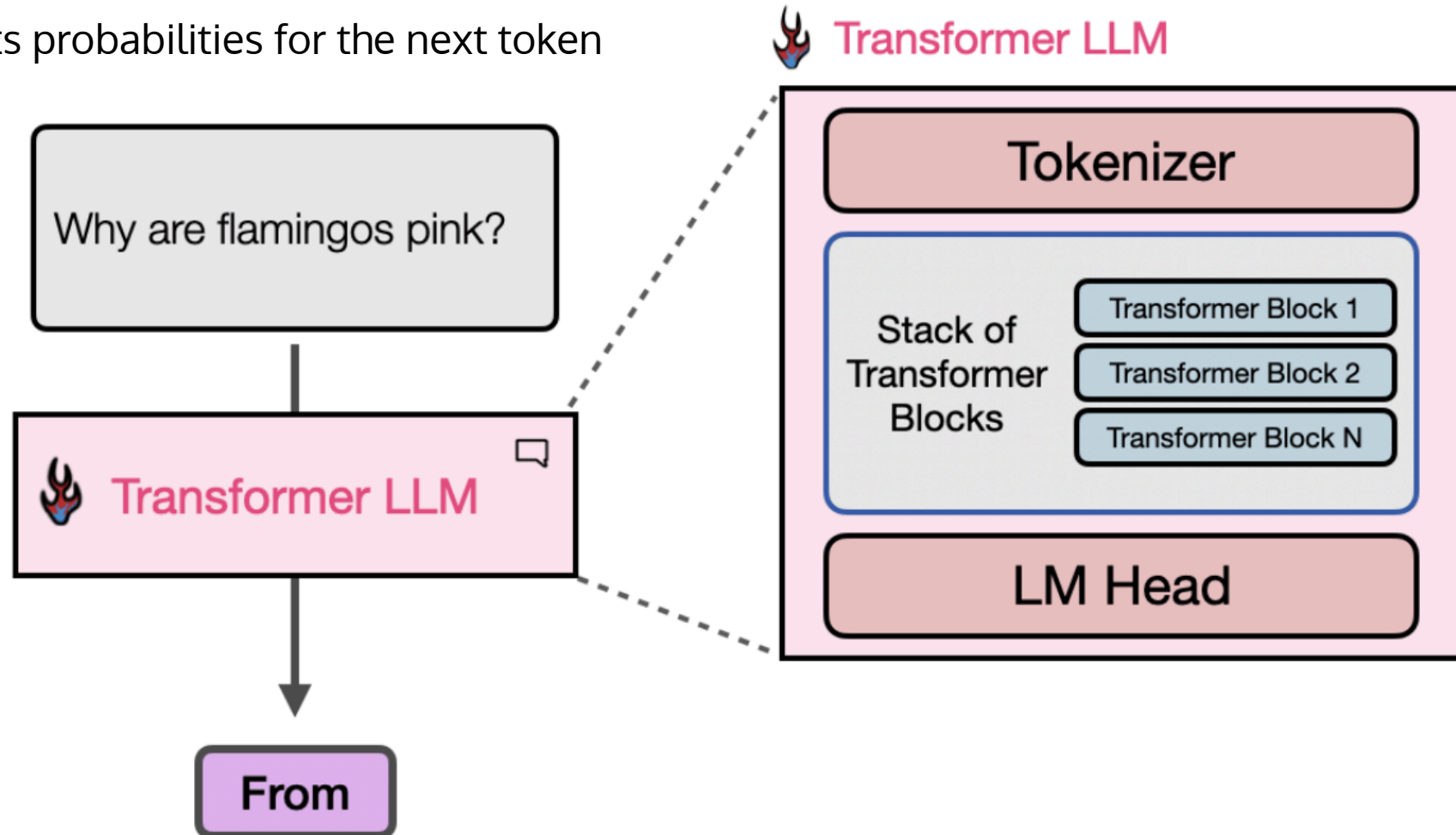
Task of a base LLM



Large language models (LLMs) are models that can generate human-like text by predicting the **probability** of a word given the previous words used in a sentence.

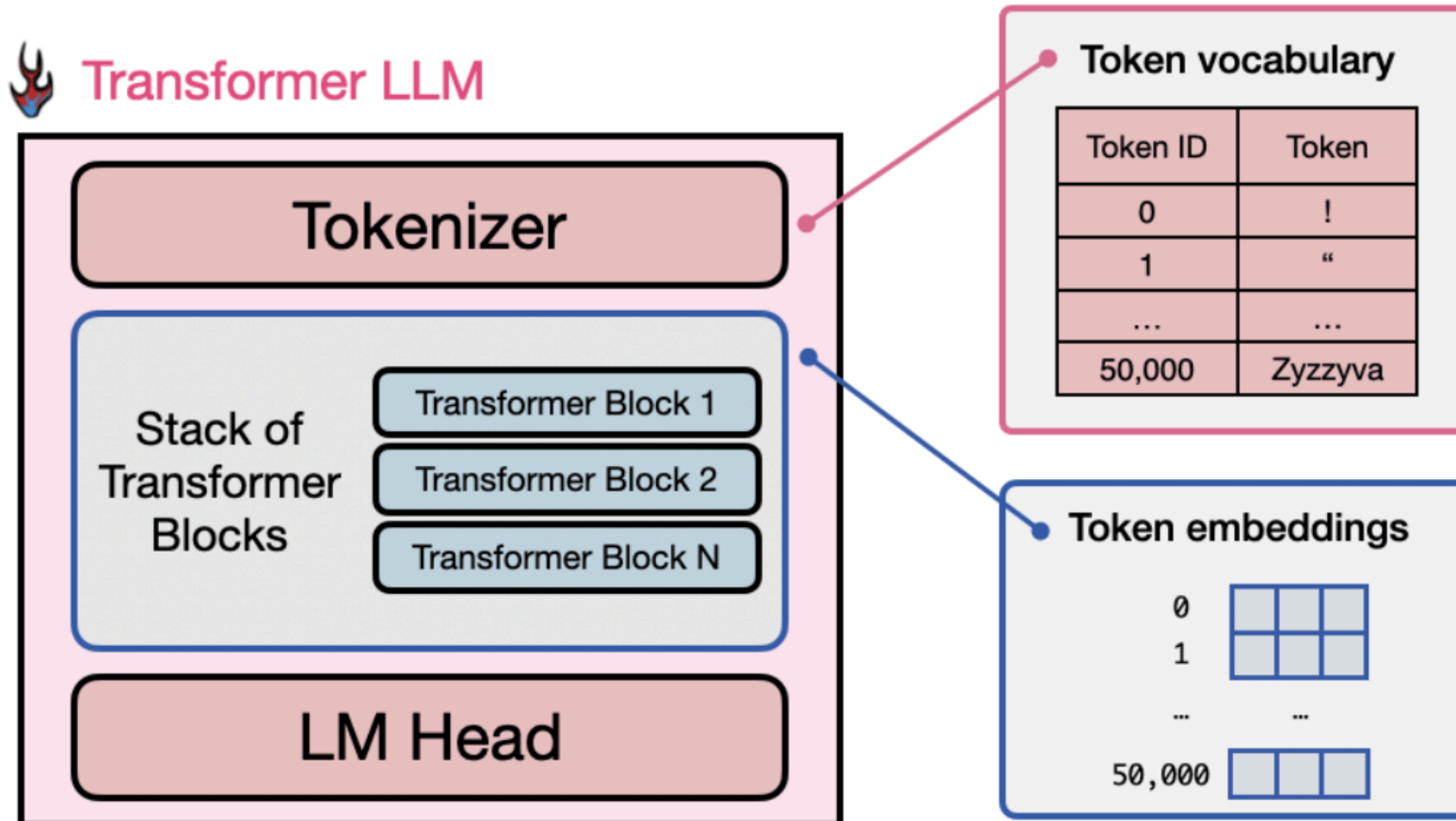
Architecture of a base LLM

- **Tokenizer:** converts text into tokens
- **Transformer blocks:** process token context
- **LM head:** predicts probabilities for the next token



Architecture of a base LLM: Words to numbers

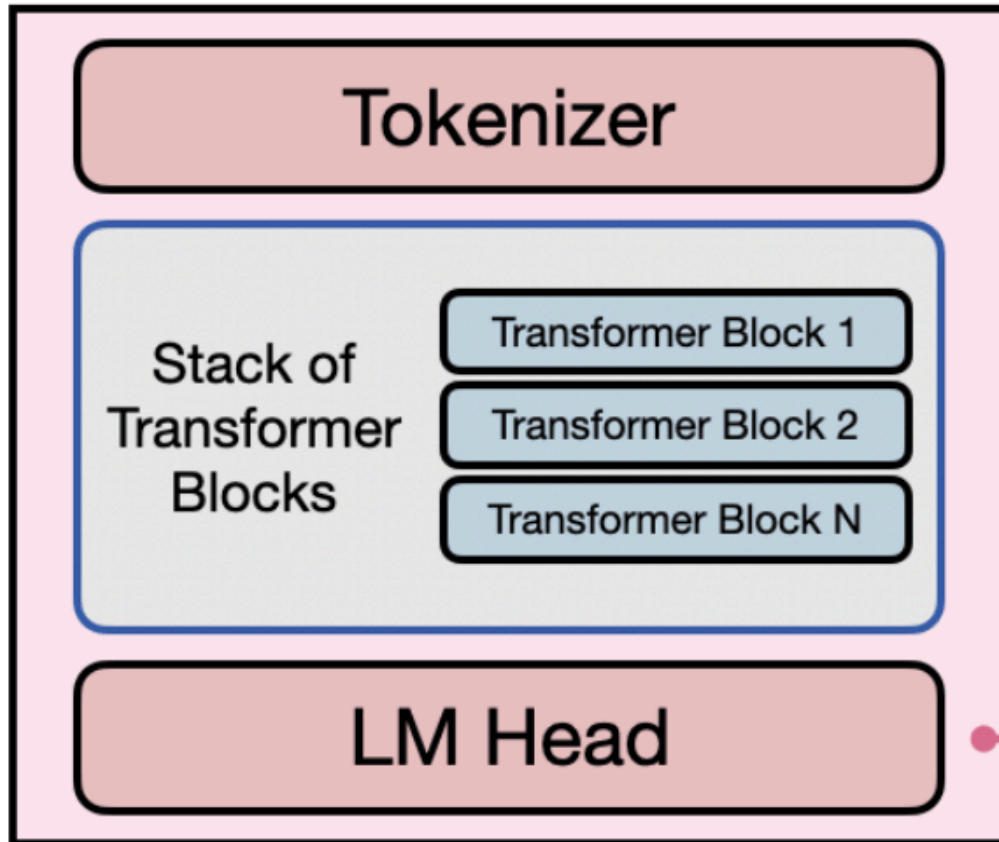
- A tokenizer has a set number of tokens in its *vocabulary*, say 50,000
- **Tokenizer**: maps text to token IDs from a fixed vocabulary
- **Embedding lookup**: converts each token ID into a numeric vector



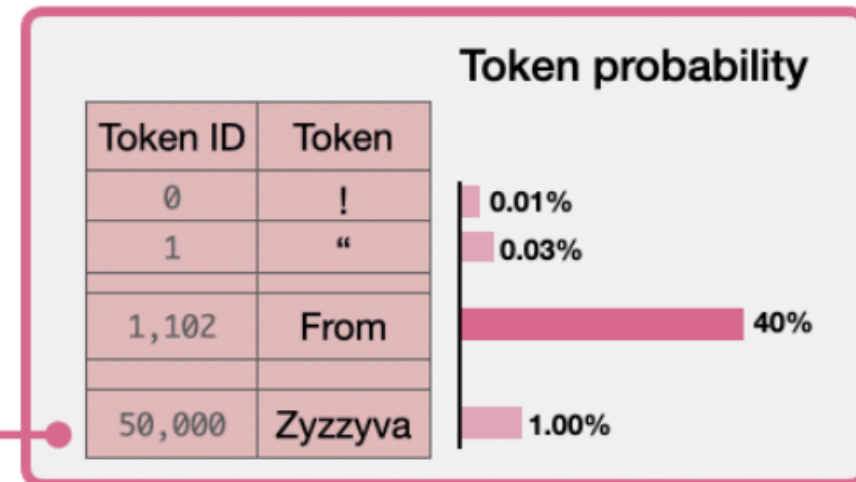
Architecture of a base LLM: Output Probabilities

- **LM head:** converts the Transformer's final token representations into next-token probabilities across the *full* vocabulary

Transformer LLM

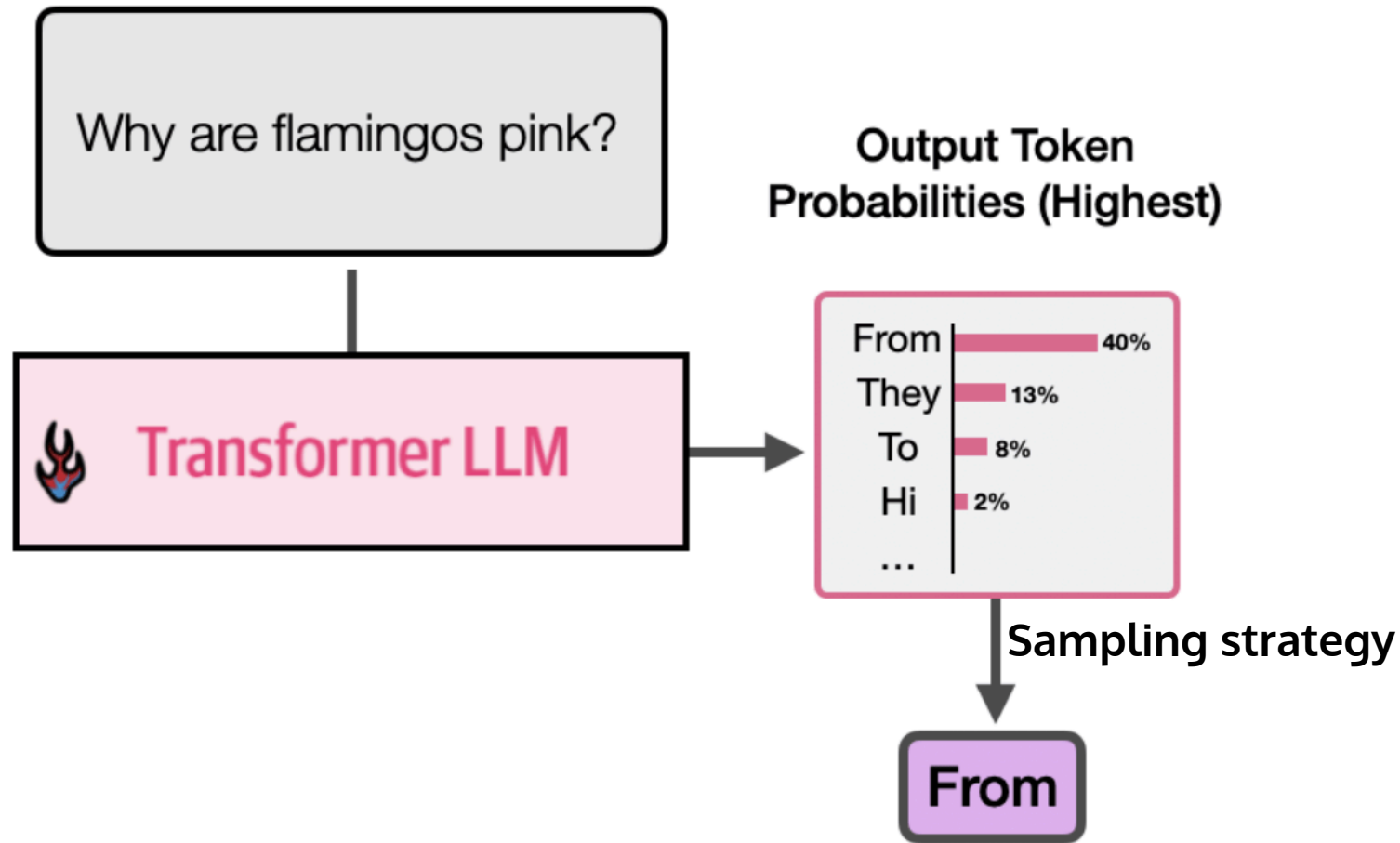


Output



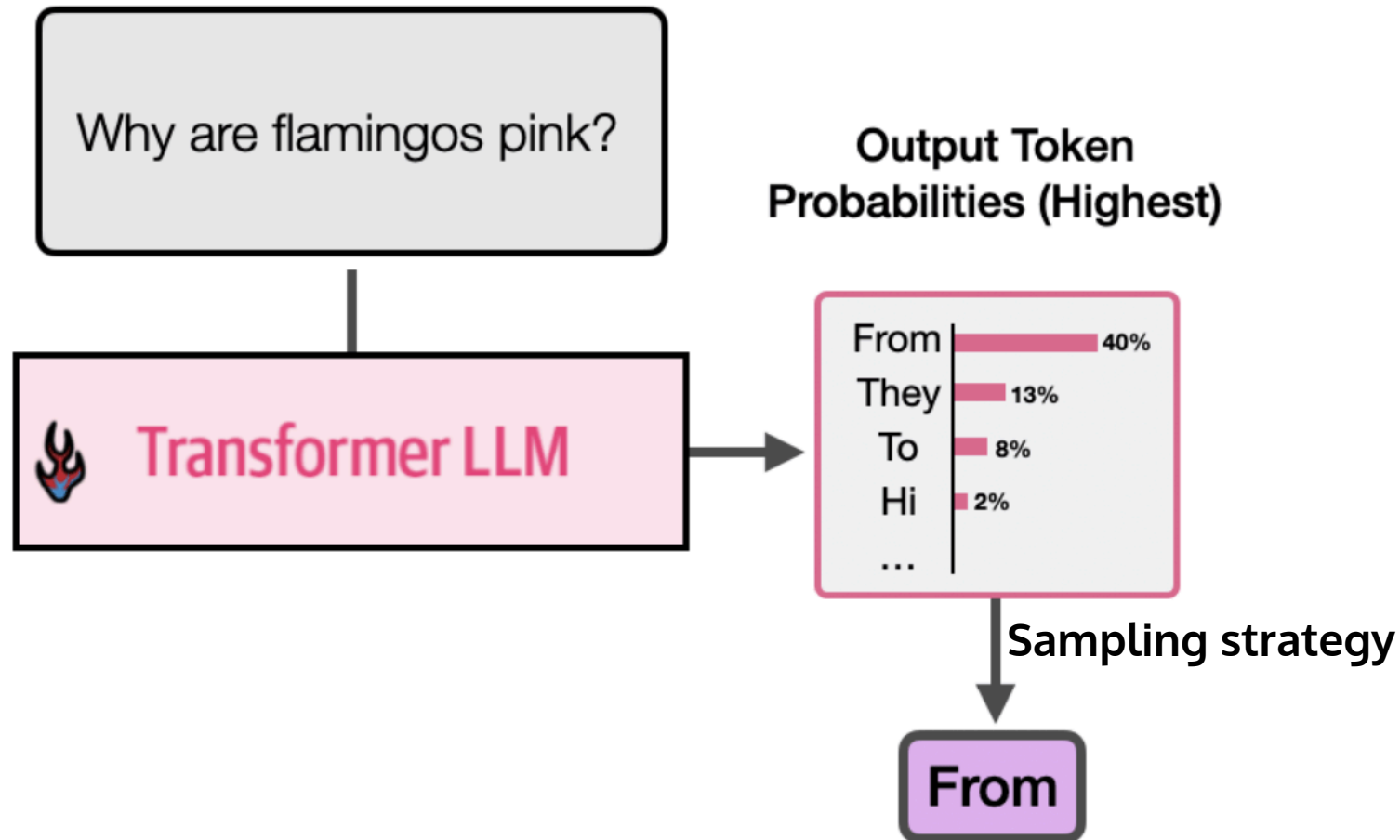
Temperature for sampling

- **Temperature = 0:** always choose the most likely token
- **Higher temperature:** sample from multiple possible tokens
- **Higher-probability tokens** remain more likely to be selected



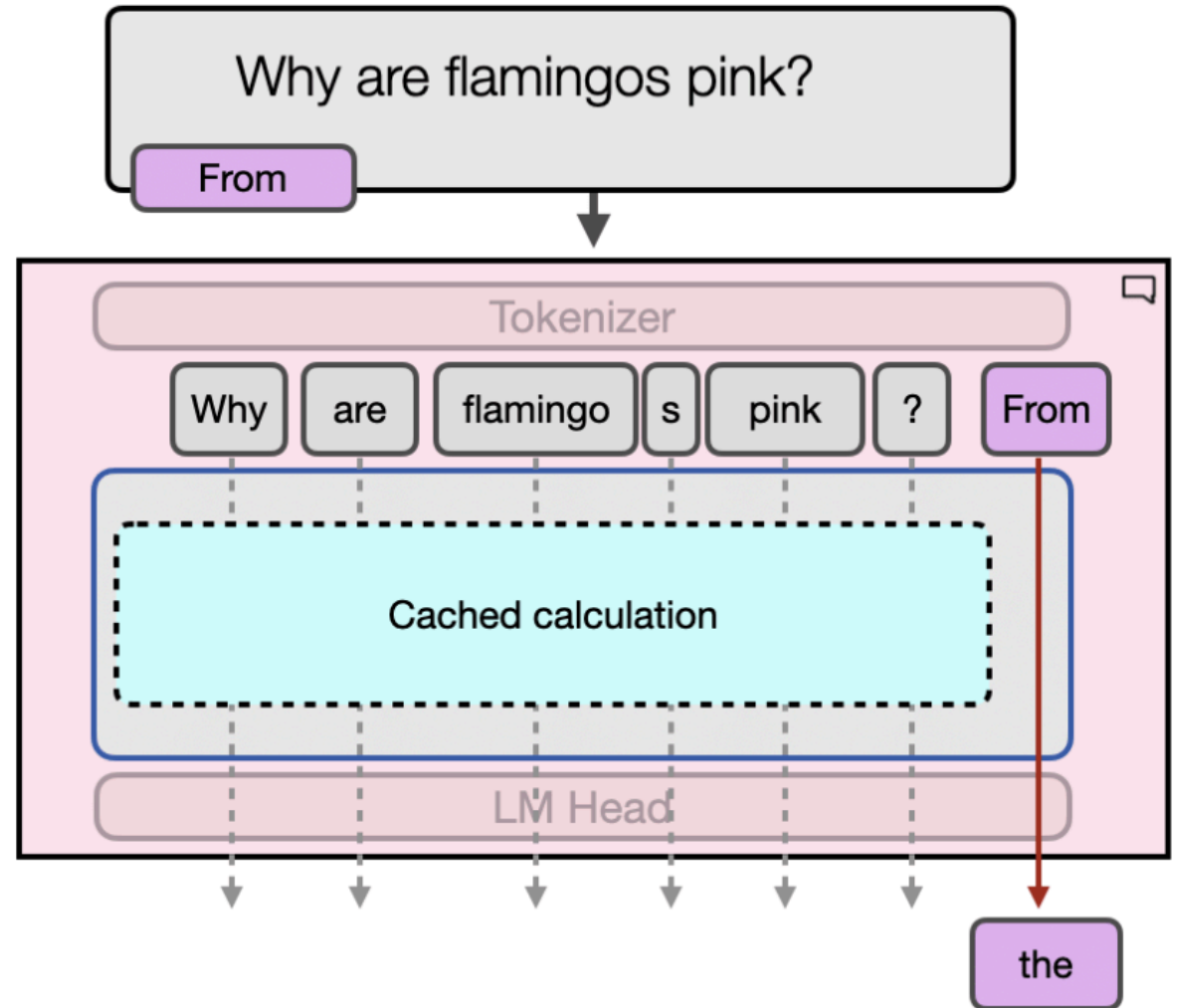
Temperature for sampling

- **Temperature = 0:** always choose the most likely token
- **Higher temperature:** sample from multiple possible tokens
- **Higher-probability tokens** remain more likely to be selected



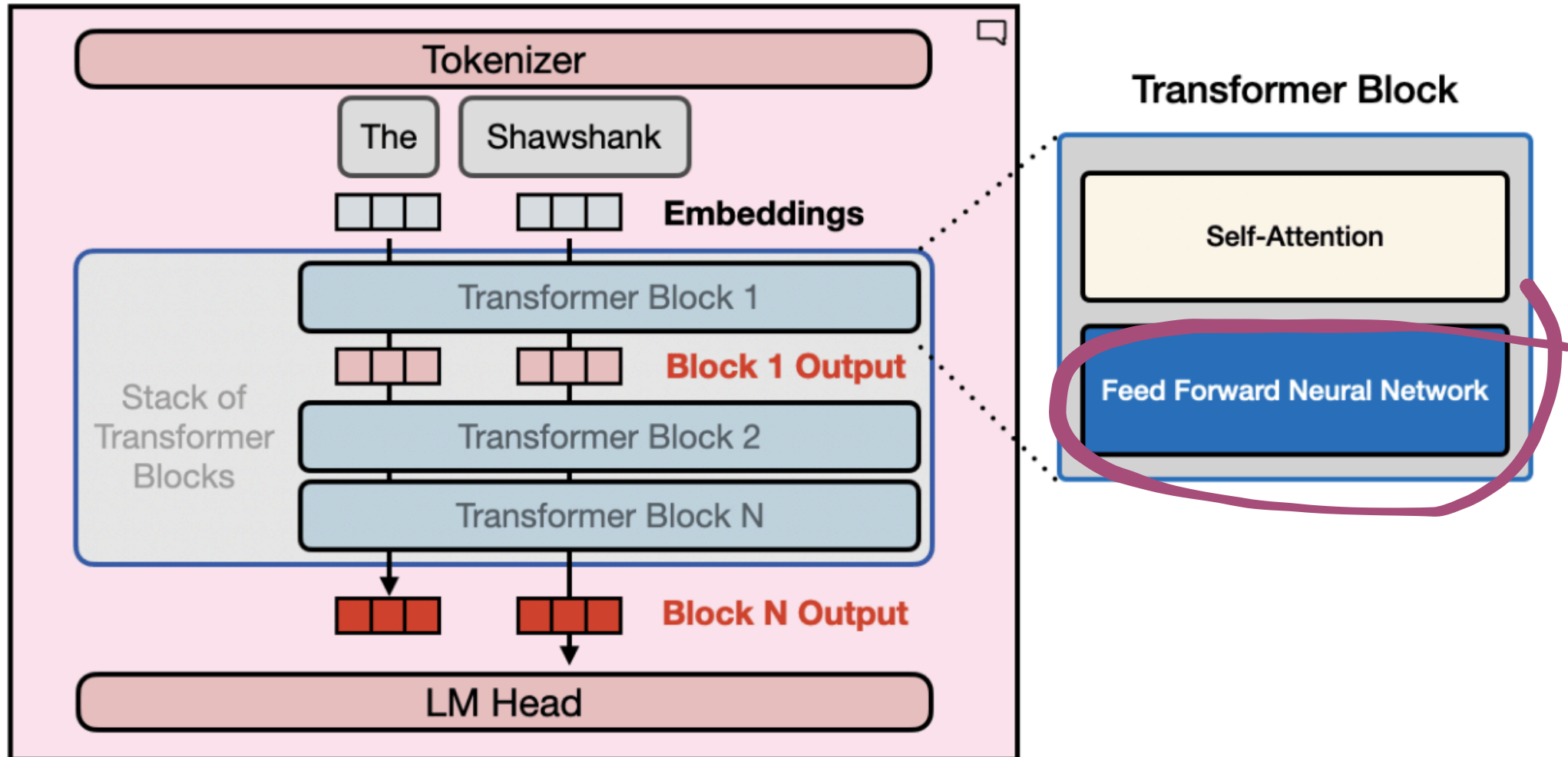
Prompt caching

- **First pass:** process the full prompt and cache attention results
- **Later tokens:** reuse the cache; process only the new token
- **Prompt cache:** makes generation much faster



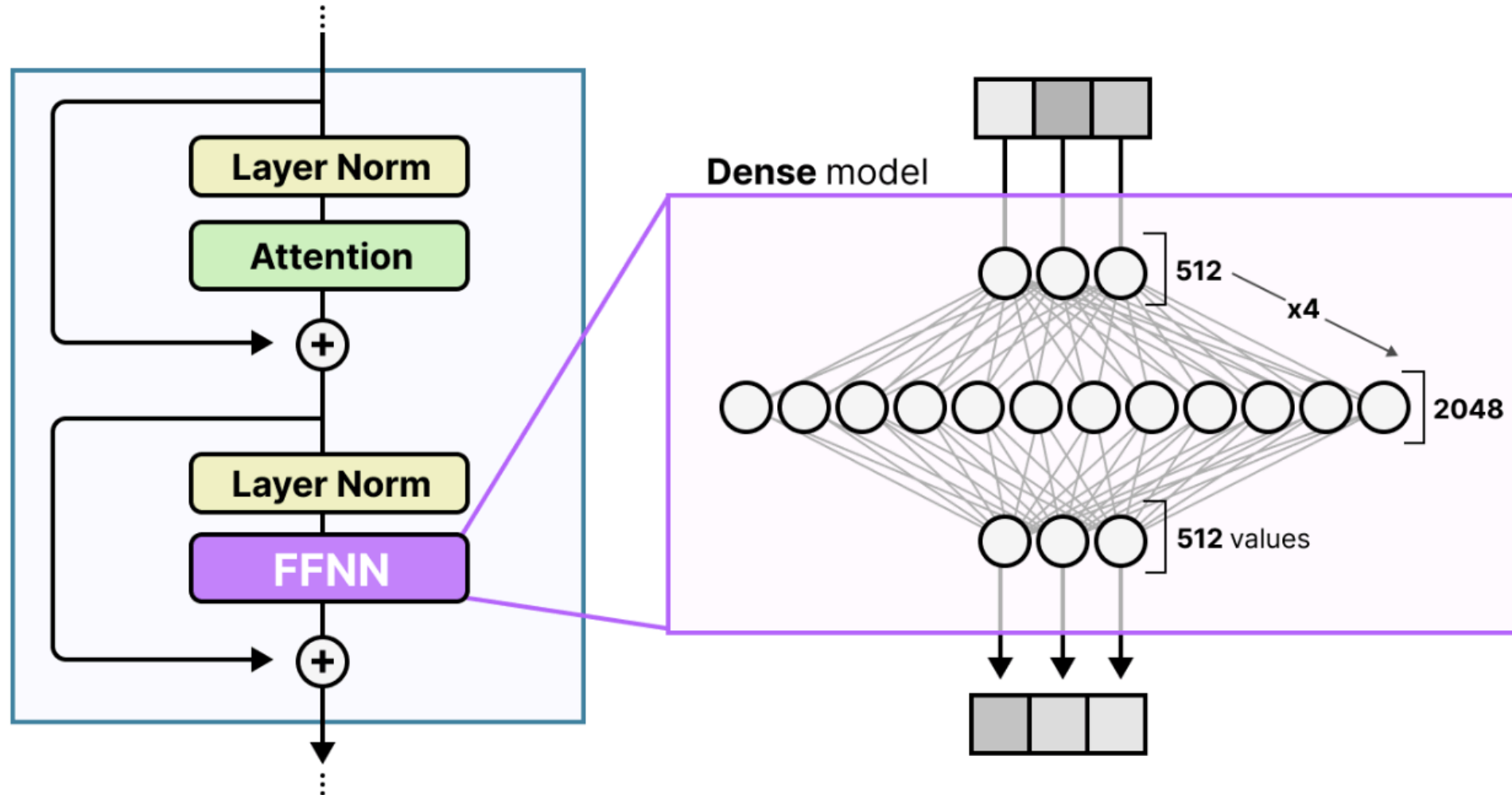
Looking inside the Transformer blocks

- **Self-attention:** each token gathers relevant context from other tokens
- **Feed-forward neural network:** independently refines each token's representation

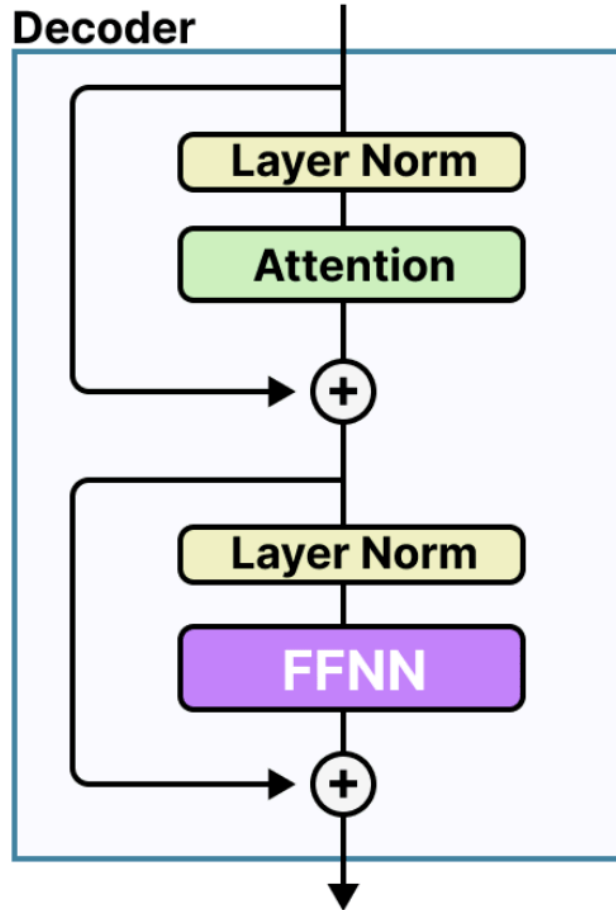


Transformer blocks: FFNN

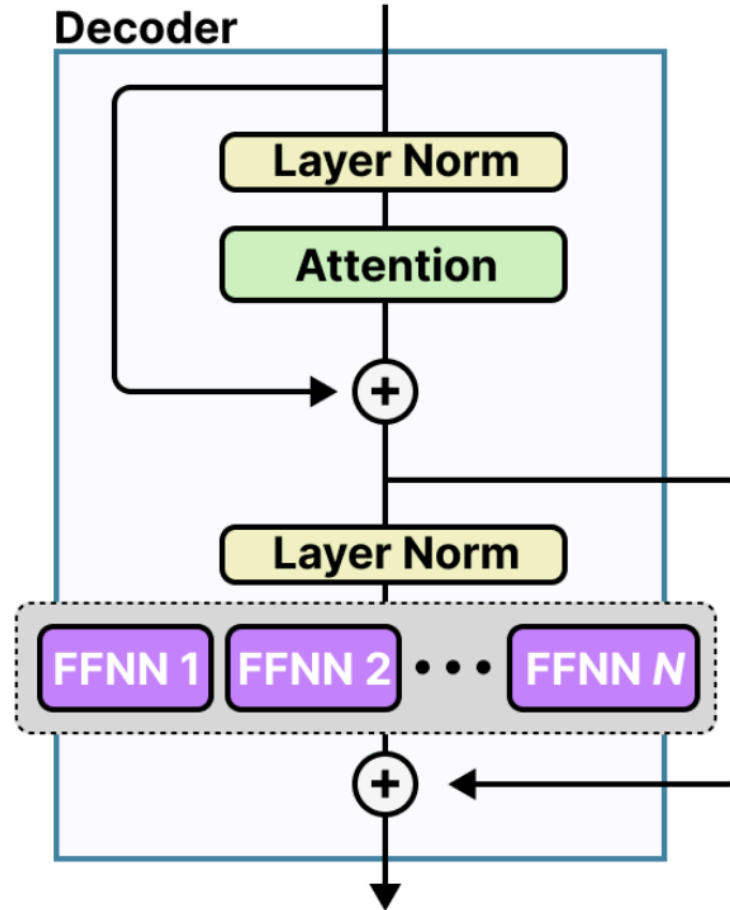
- **Dense FFNN model:** every token uses the same full feed-forward network



Dense vs Sparse Transformer blocks



Dense model

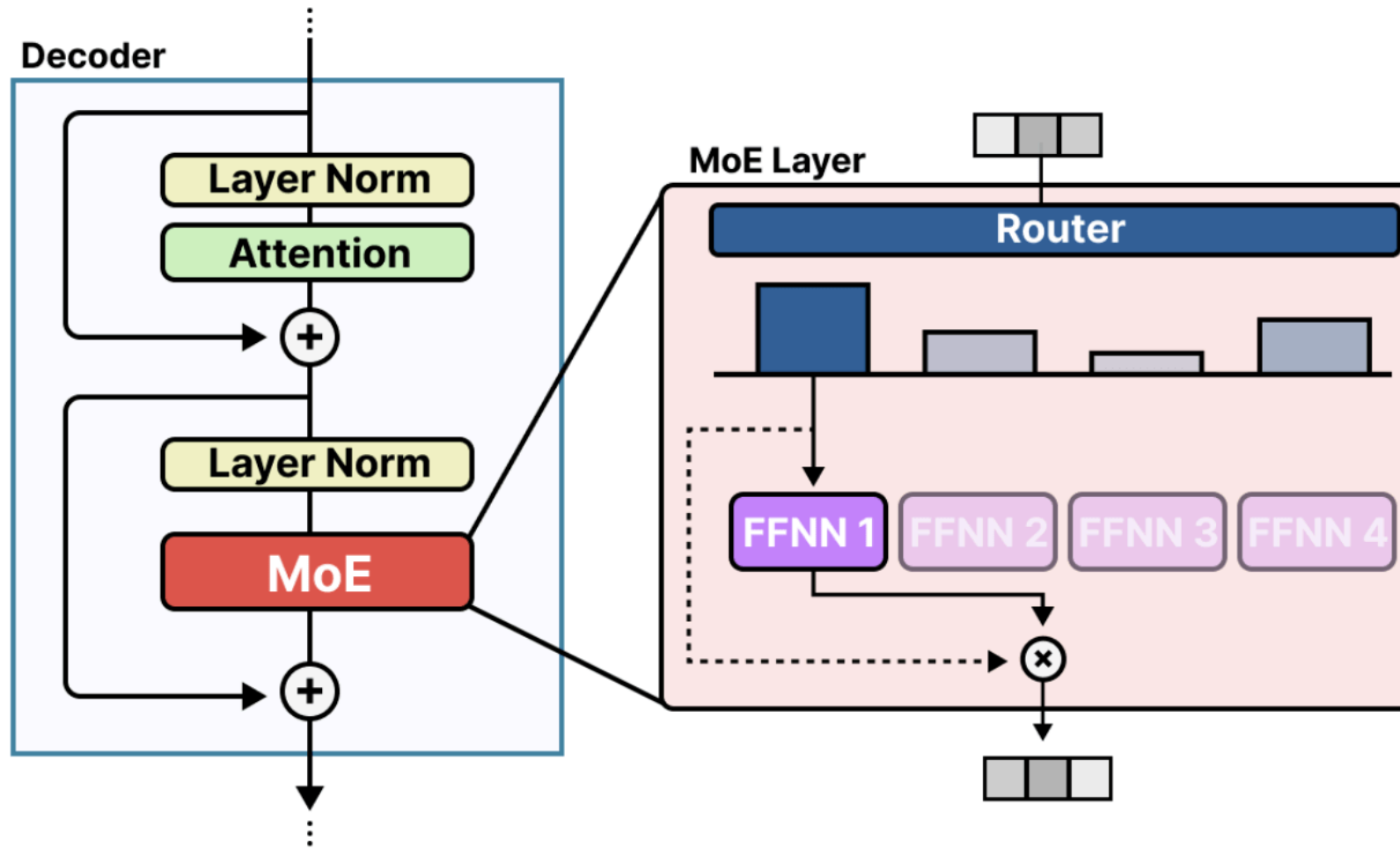


Sparse model

Benefit: higher total model capacity without using all parameters for every token

Sparse Transformer blocks: MoE

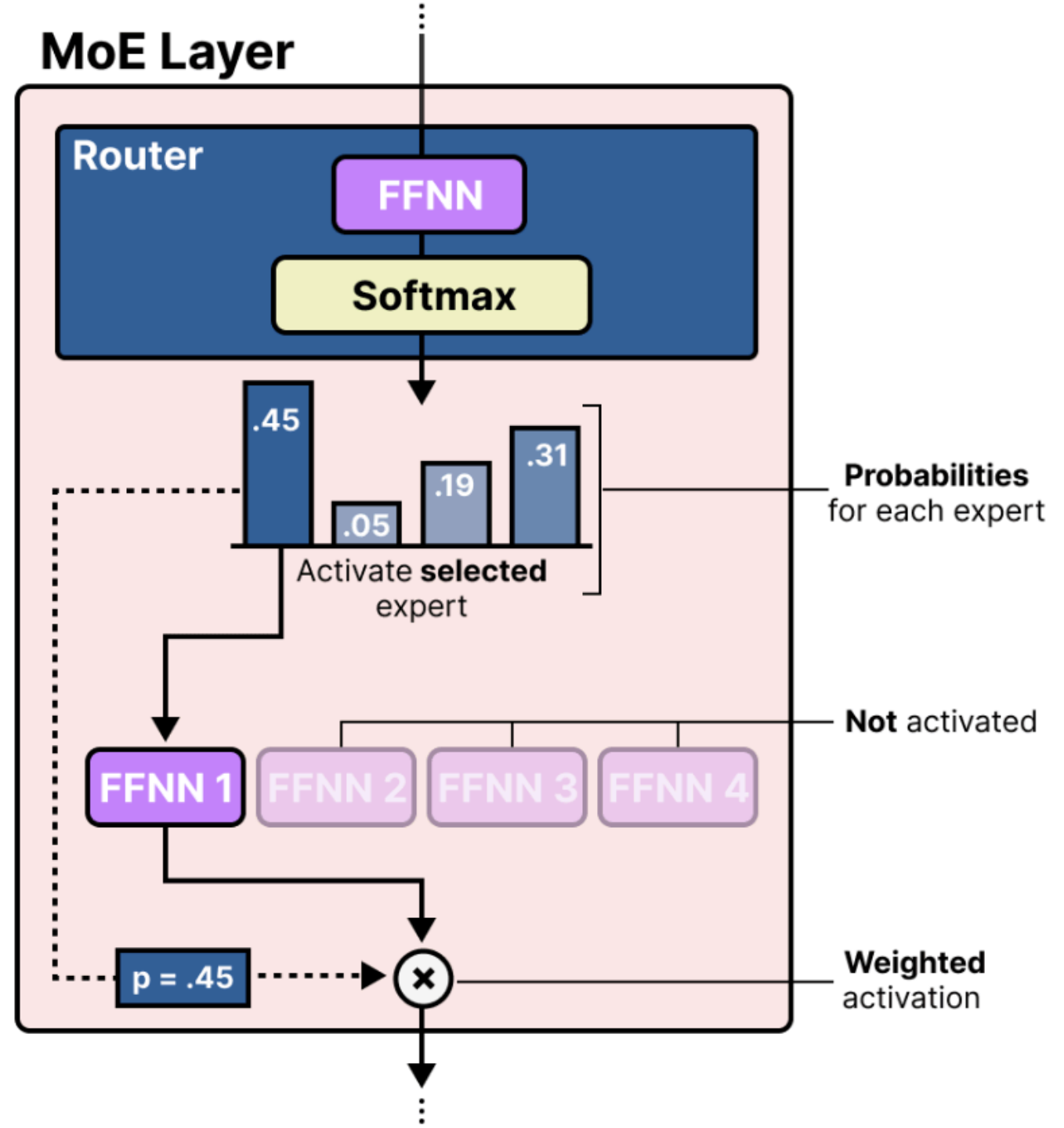
- **Mixture of Experts (MoE):** replaces one large network with many smaller expert networks
- A **router** selects the most relevant expert(s) for each token
- Only selected experts are activated; others remain inactive



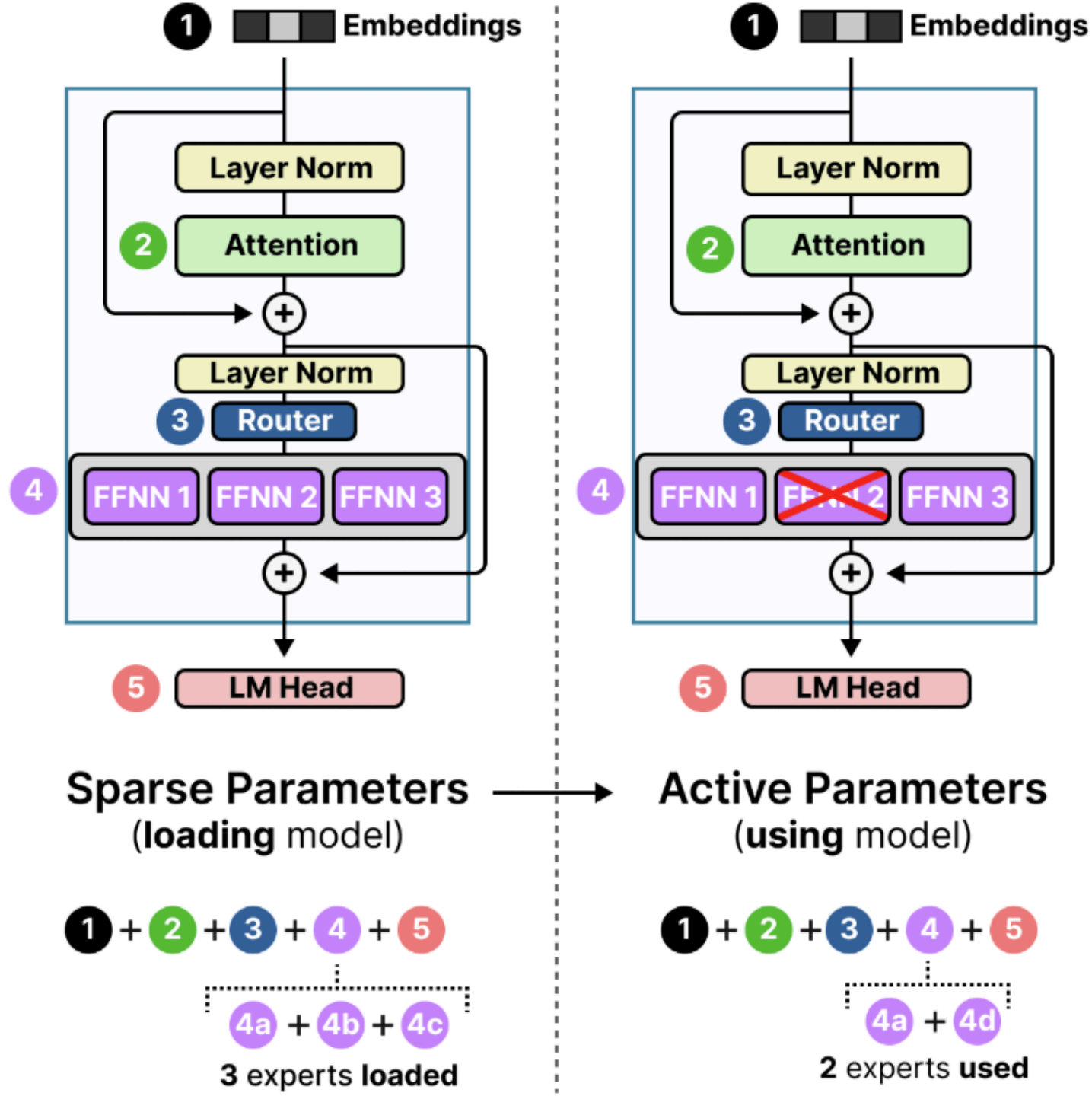
Inside the Router

- **Router network:** applies learned weights to each token representation
- **Softmax output:** gives a probability for each expert
- **Routing decision:** experts with the highest probabilities are selected for that token

MoE makes computations faster

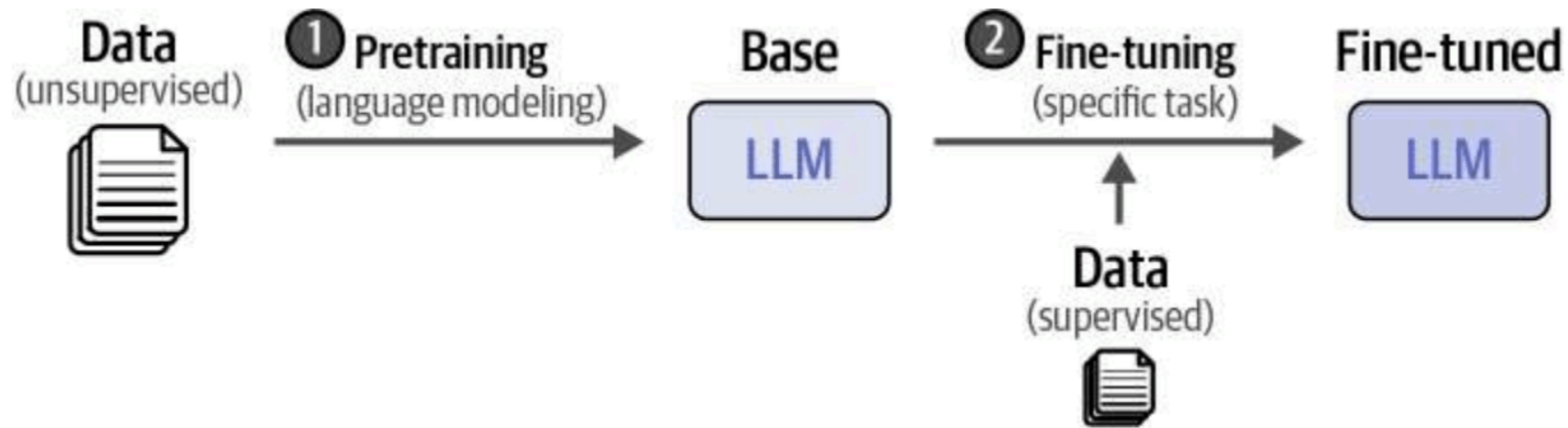


- All experts are loaded in memory
- Only selected experts are **active** for each token
- Many recent LLMs use **Mixture-of-Experts (MoE)** layers
 - [GPT-OSS](#)
 - Qwen3-30B-A3B
 - DeepSeek-R1
- Model names may show **total (sparse)** vs. **active** parameters
- Example: Qwen3-30B-A3B has **30B total parameters**, **3B active per token**



Training an LLM

LLMs go through two major phases of training: **pre-training** and **post-training**



Pre-training an LLM

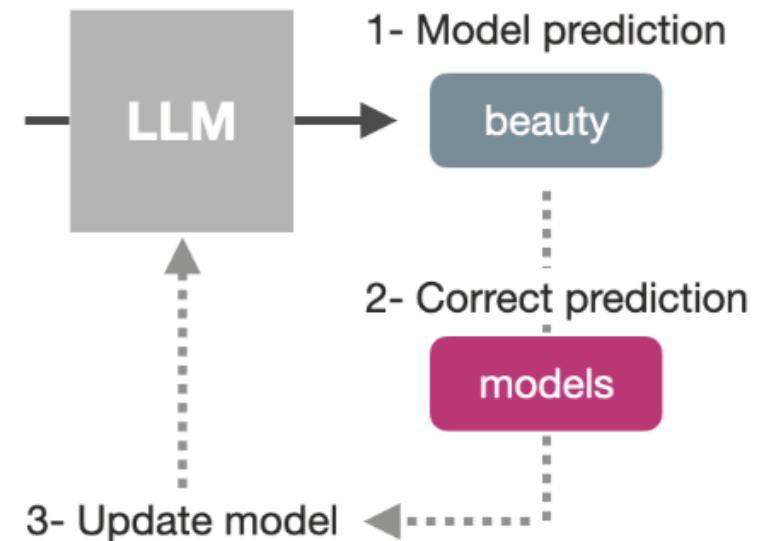
Pre-training as next-token prediction

- Understand the architecture of Transformer language models that excel at text generation and representation
- Build advanced LLM pipelines to cluster text documents and explore the topics they cover
- Build semantic search engines that go beyond keyword search, using methods like dense retrieval and rerankers
- Explore how generative models can be used, from prompt engineering all the way to retrieval-augmented generation
- Gain a deeper understanding of how to train LLMs and optimize them for specific applications using generative model fine-tuning, contrastive fine-tuning, and in-context learning

“Understand the architecture of Transformer language models”

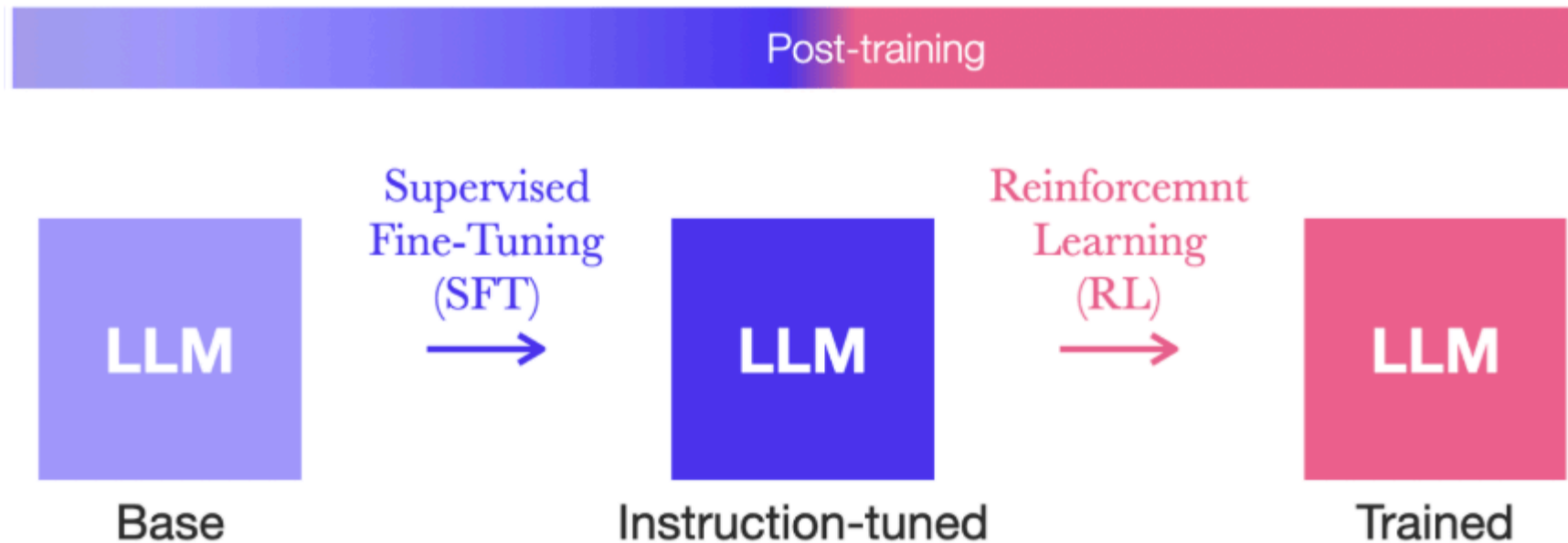
Understand the architecture of large language

Pretraining uses curated web text, books, and code filtered for quality



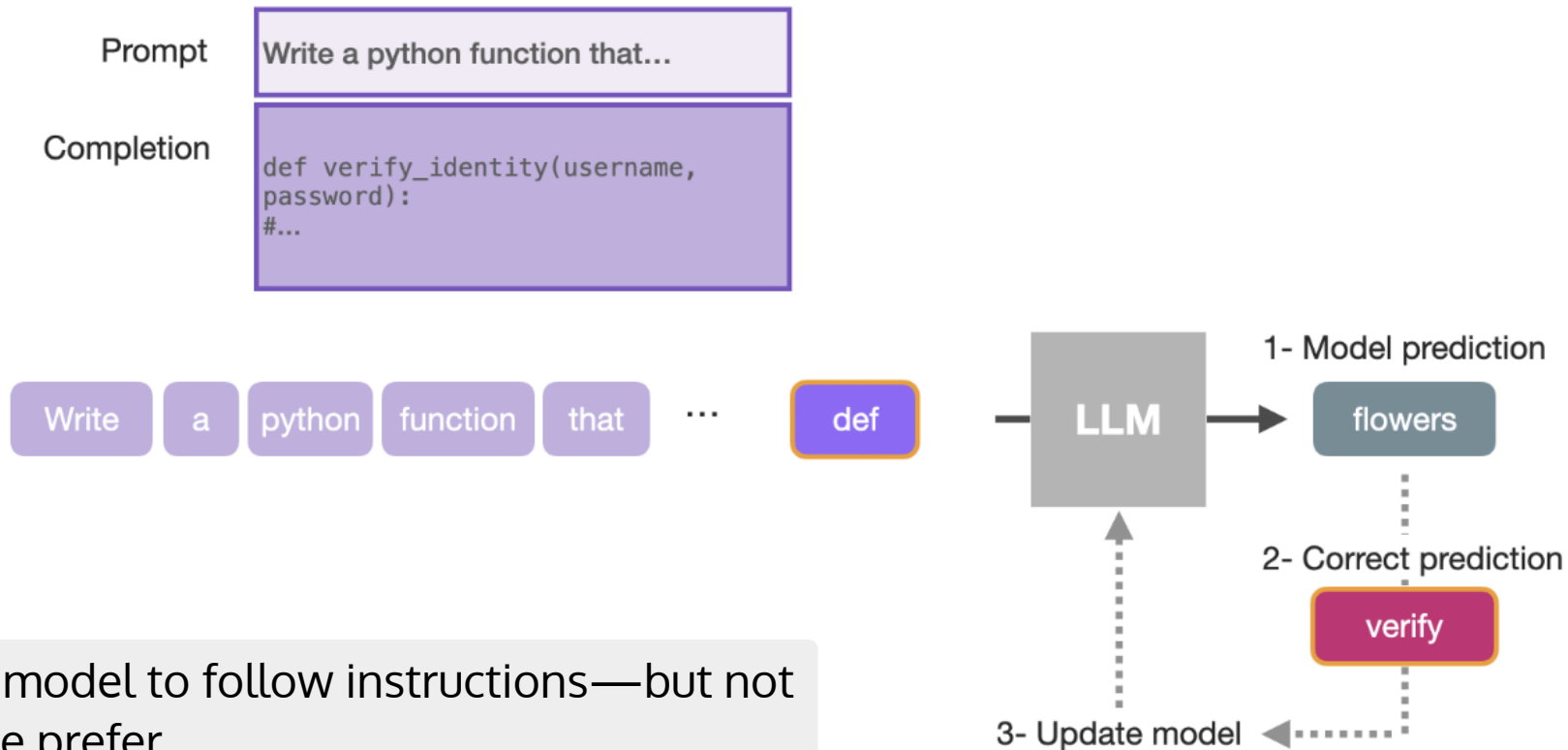
Post-training an LLM

- Base models have strong latent capabilities: reasoning, summarization, translation, and code generation
- But they often require heavy prompt engineering to unlock
- Post training helps polish these model behaviors using a much smaller set of training example



Post-training an LLM: Supervised Fine-Tuning

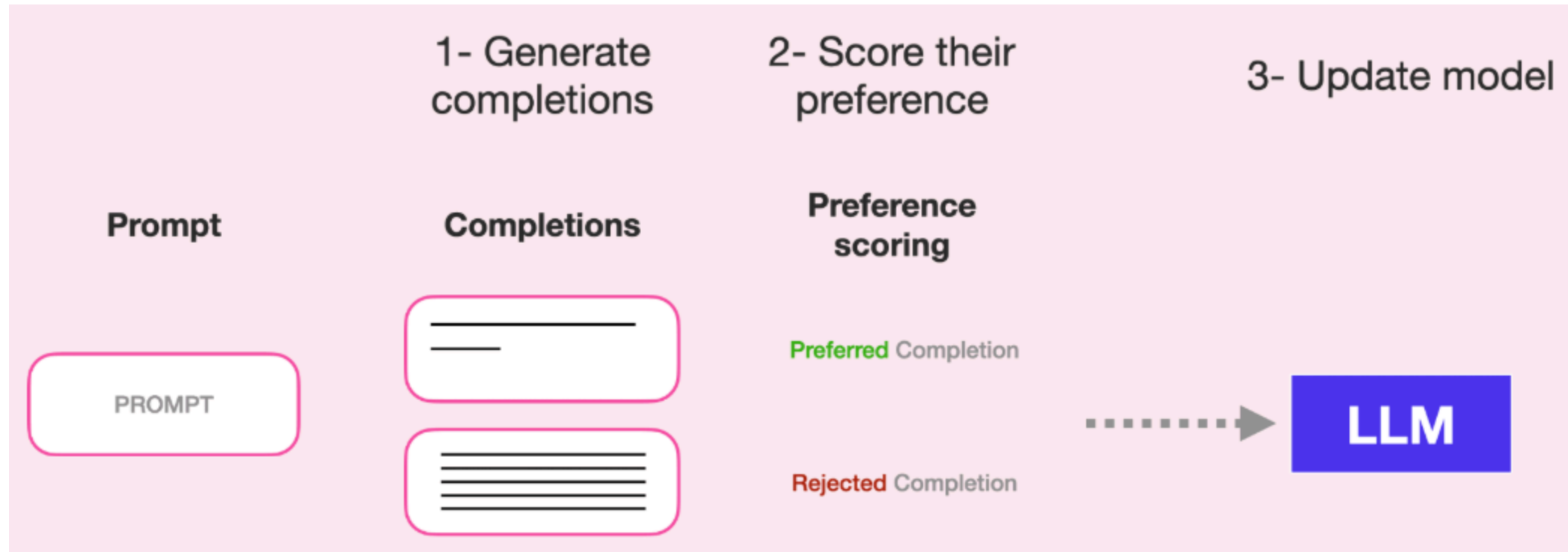
- **SFT examples:** prompt + ideal completed response
- **Training rule:** same basic next-token learning as pretraining
- **Key difference:** prompt provides context, but loss is computed only on the response tokens



SFT teaches the model to follow instructions—but not fully what people prefer

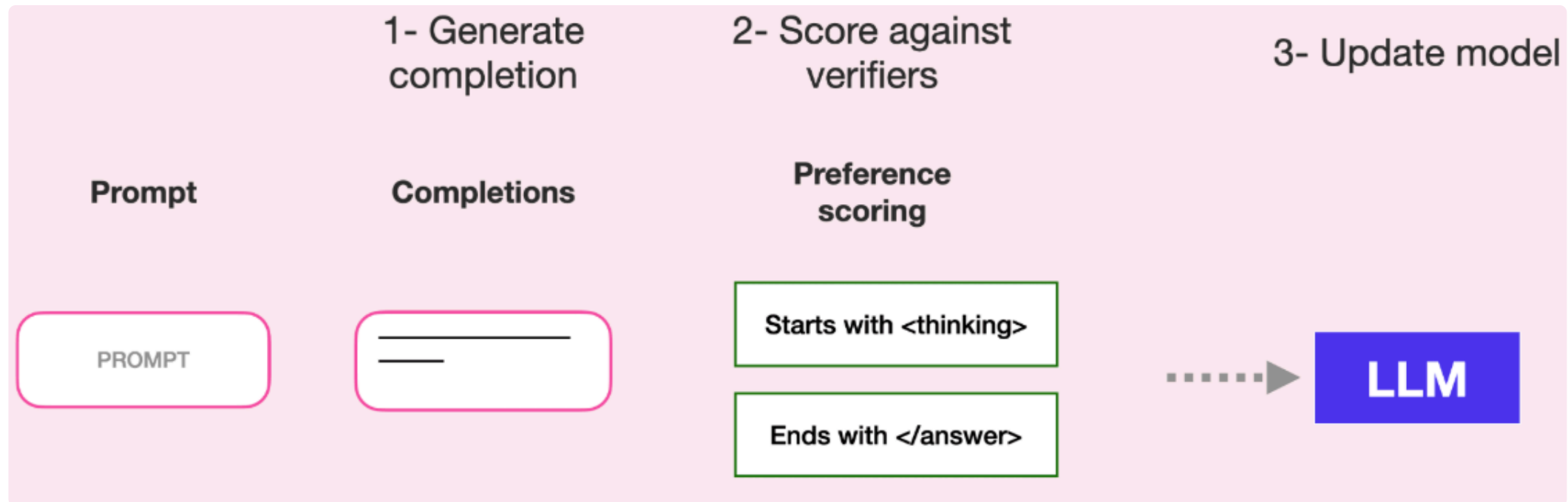
Post-training an LLM: Reinforcement Learning

- **Reinforcement Learning with Human Feedback (RLHF)** uses **preference feedback** from humans
- The model learns which response is **preferred** and which is **rejected**
- Training increases the likelihood of preferred outputs and reduces rejected ones



Post-training an LLM: Reinforcement Learning

- **RLVR: Reinforcement Learning with Verifiable Rewards**
- Like RLHF, it improves the model using reward scores
- But rewards come from **automatic checks**, not human preferences
- Examples: correct math answer, valid code execution, required output format
- Useful when correctness can be objectively verified



Post-training an LLM: Reinforcement Learning

RLVR loop: prompt → answer → verify format + correctness → reward → update model

RL training dataset

	Math problem	Answer
1	If the Toronto Reference Library loans...	42
2	An animal shelter hosts 14 cats...	46
...	...	...
500	If a train departs Riyadh at 8AM...	28

Solve the following math problem. print the final answer between `<answer></answer>` tags.

System Prompt

If the Toronto Reference Library loans...

Math problem

LLM

Instruction-tuned

Let's start by calculating...

...
`<answer>0</answer>`

Reward assignment

Format reward:

0.3 if correct, 0 otherwise

is `<answer>` used?

0.3

Accuracy reward:

0.7 if correct, 0 otherwise

Was it the correct answer?

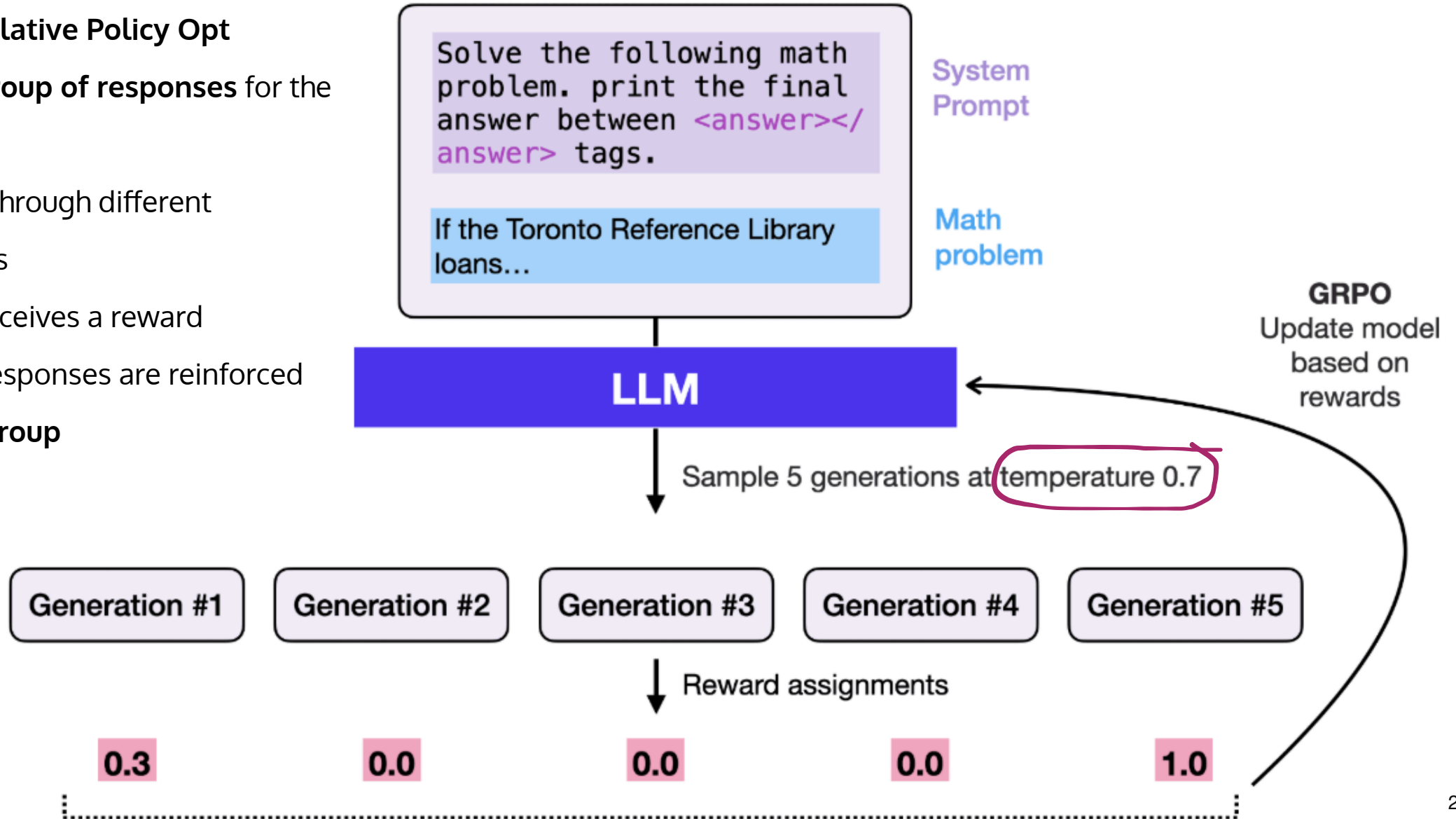
0

0.3 + 0 = 0.3 out of 1.0

RL
Update model
based on
rewards

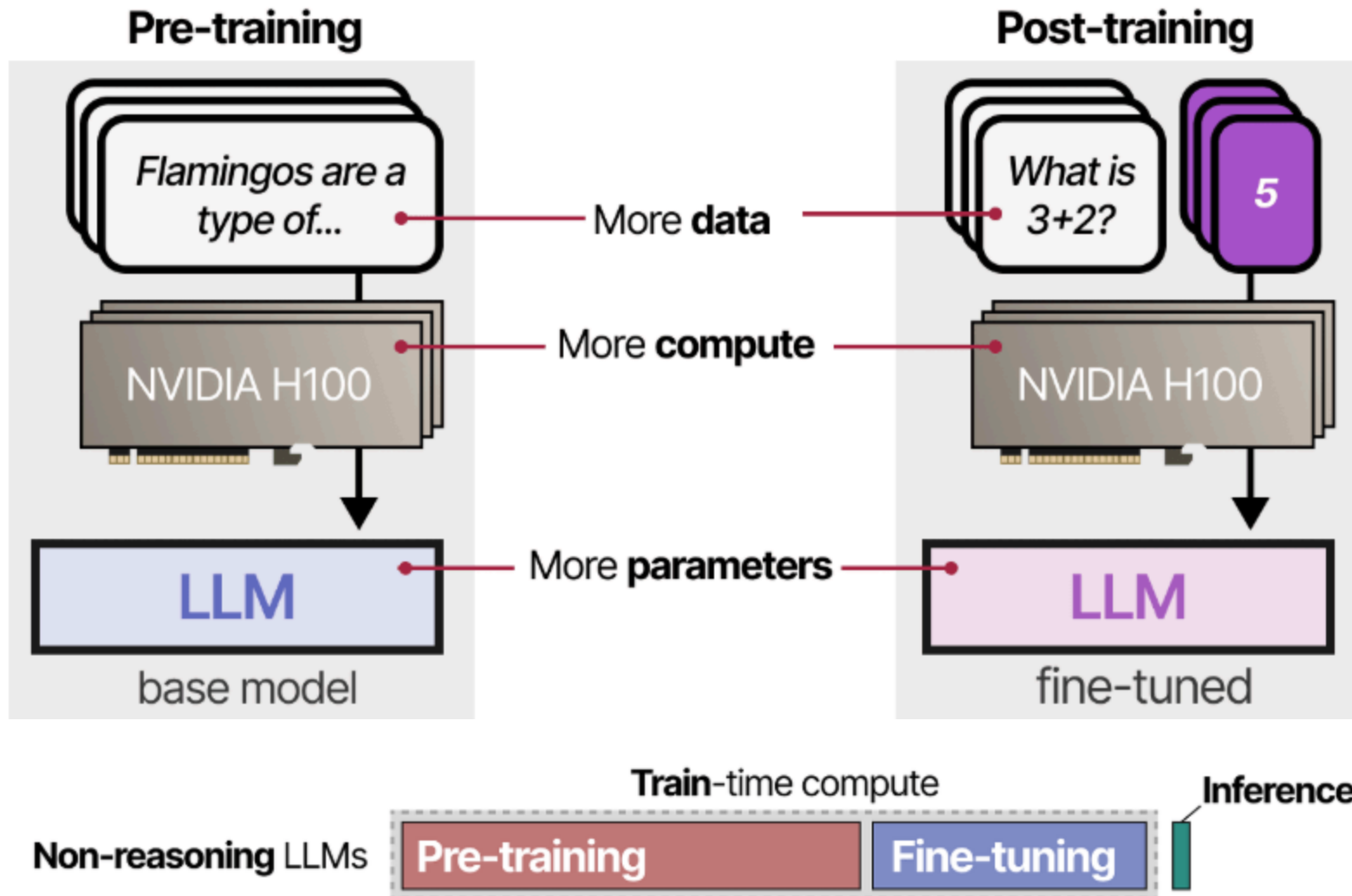
Post-training an LLM: Reinforcement Learning

- **GRPO: Group Relative Policy Opt**
- It **generates a group of responses** for the same prompt
- Responses vary through different sampling settings
- Each response receives a reward
- Higher-scoring responses are reinforced **relative to the group**



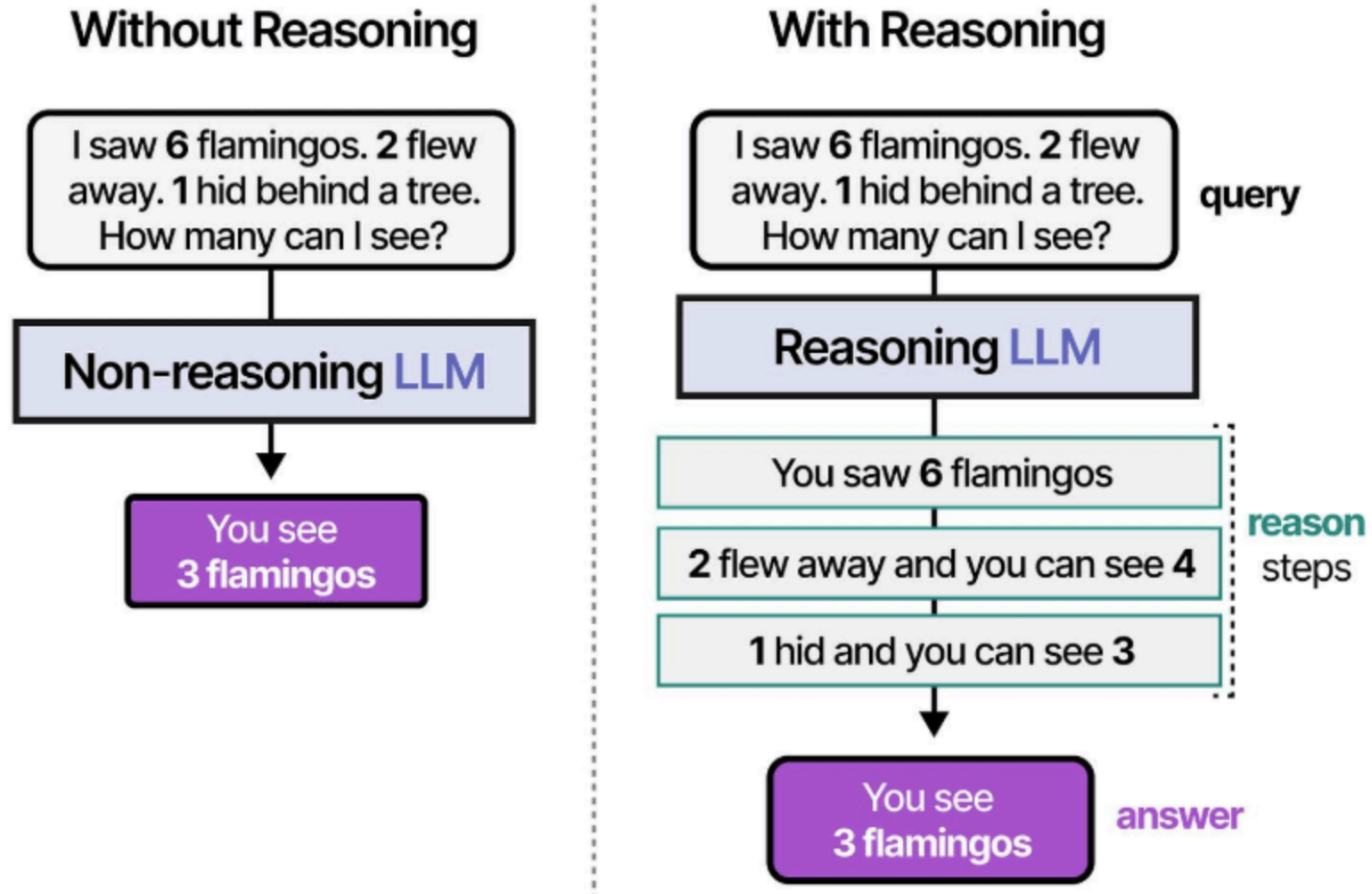
Train-time vs Test-time Compute

Train-time compute relates to the time taken for entire training process of LLMs: Pre-training + Post-training



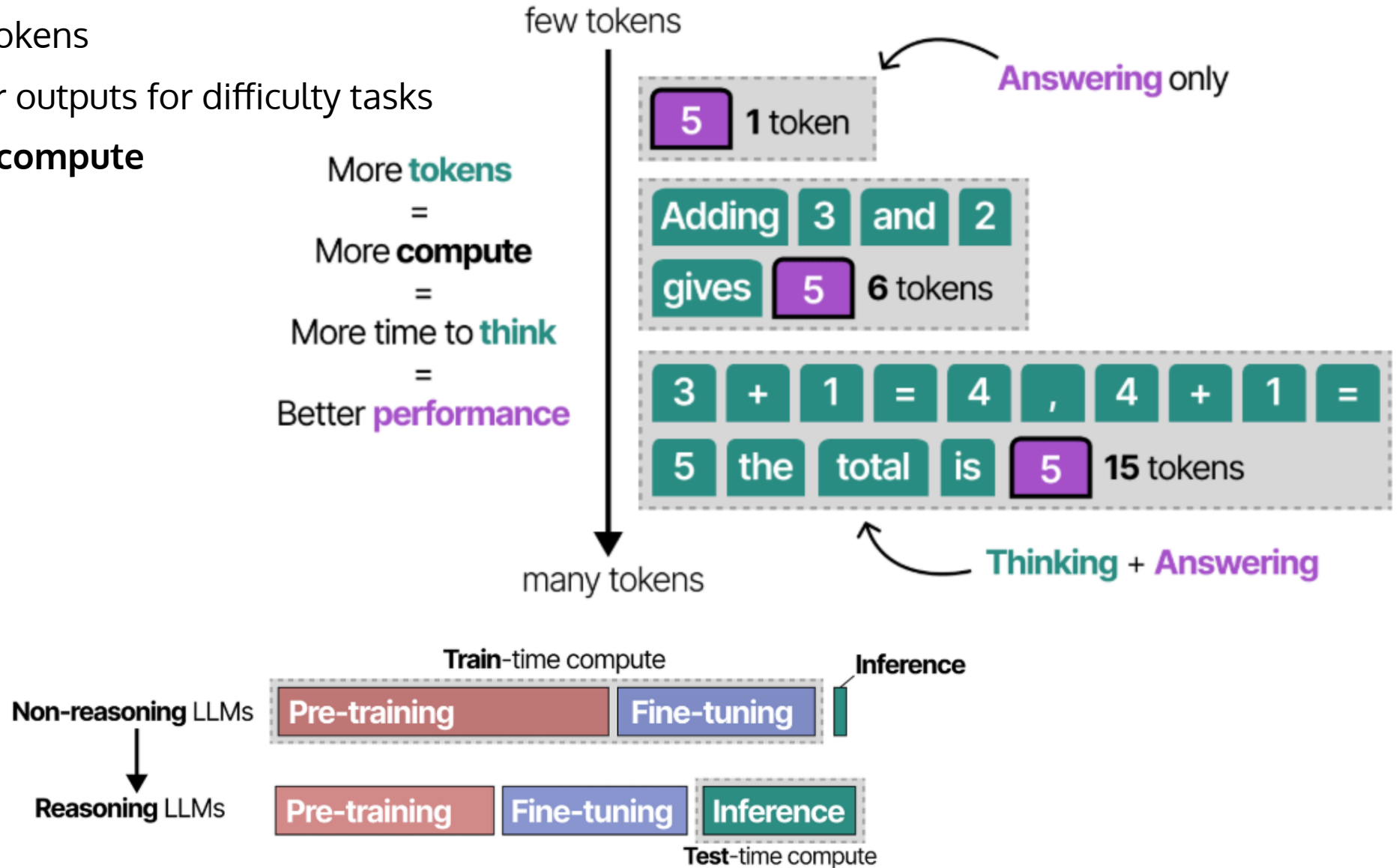
Reasoning LLMs

Reasoning in LLMs attempts to mimic our way of thinking by generating thoughts through tokens before giving a final answer



Reasoning LLMs

- They consume more tokens
- But they deliver better outputs for difficulty tasks
- Take **more test-time compute**



How to reason during test-time?

Two ways to improve reasoning at test time

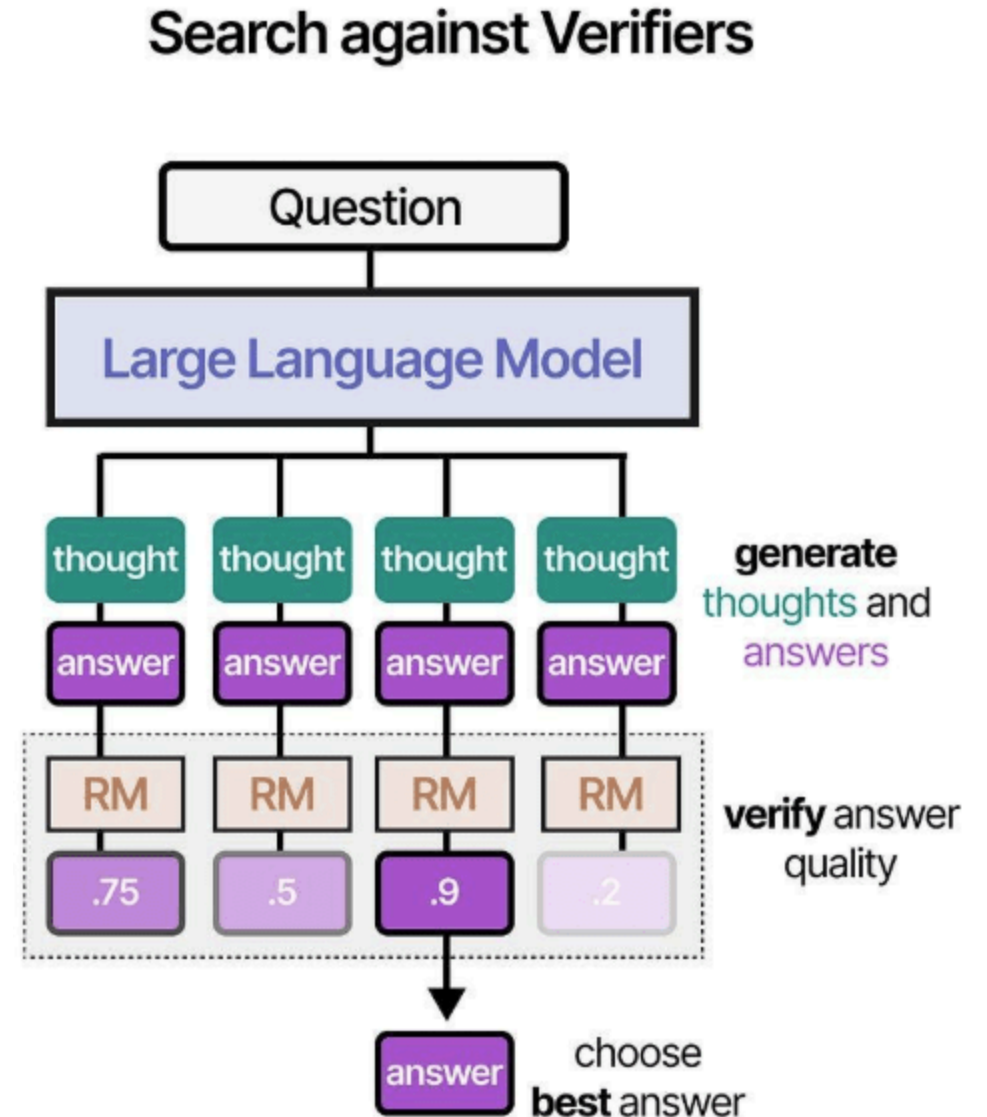
1. Generate more candidate solutions and verify them : **Search against verifiers**
2. Make the model generate better reasoning traces in the first place : **Modifying Proposal Distribution**

A reasoning system can either **search harder** among candidate solutions, or **think better** when producing each candidate

How to reason during test-time?

Search against verifiers

- **Idea:** Generate many candidate answers or reasoning traces, then use a **verifier or reward model (RM)** to select the best one
- It is **output-focused** as it improves the answer by searching among alternatives

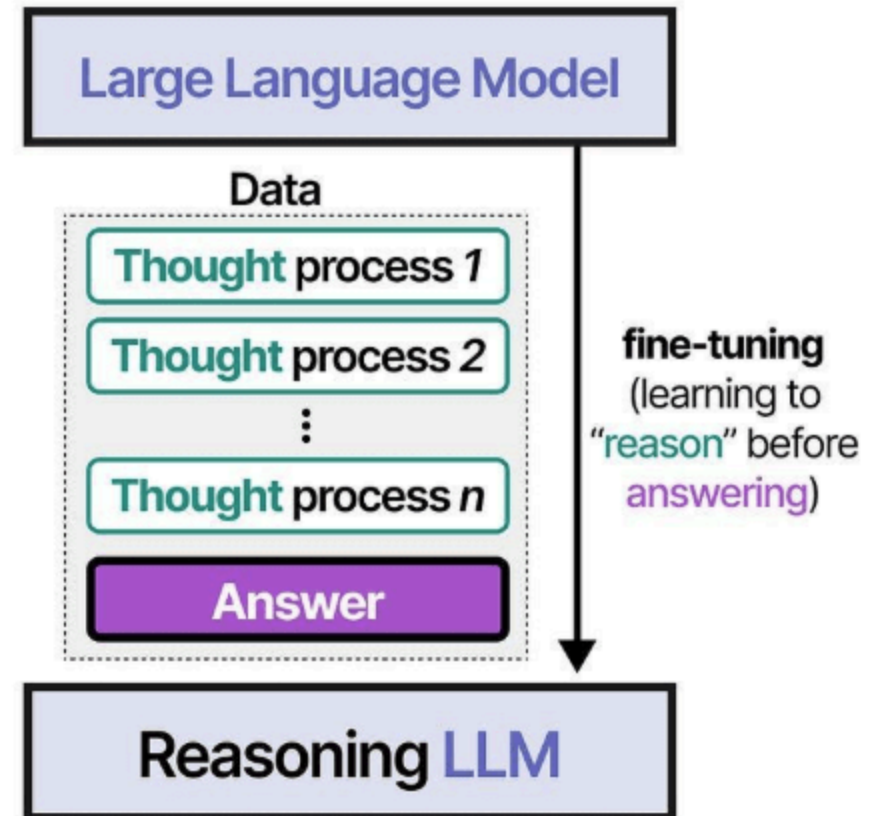


How to reason during test-time?

Modifying Proposal Distribution

- **Idea:** Train or prompt the model so that its next-token probabilities produce better reasoning traces from the start
- It is **input-focused** as it improves how the model proposes its next steps

Modifying Proposal Distribution



Reward Models

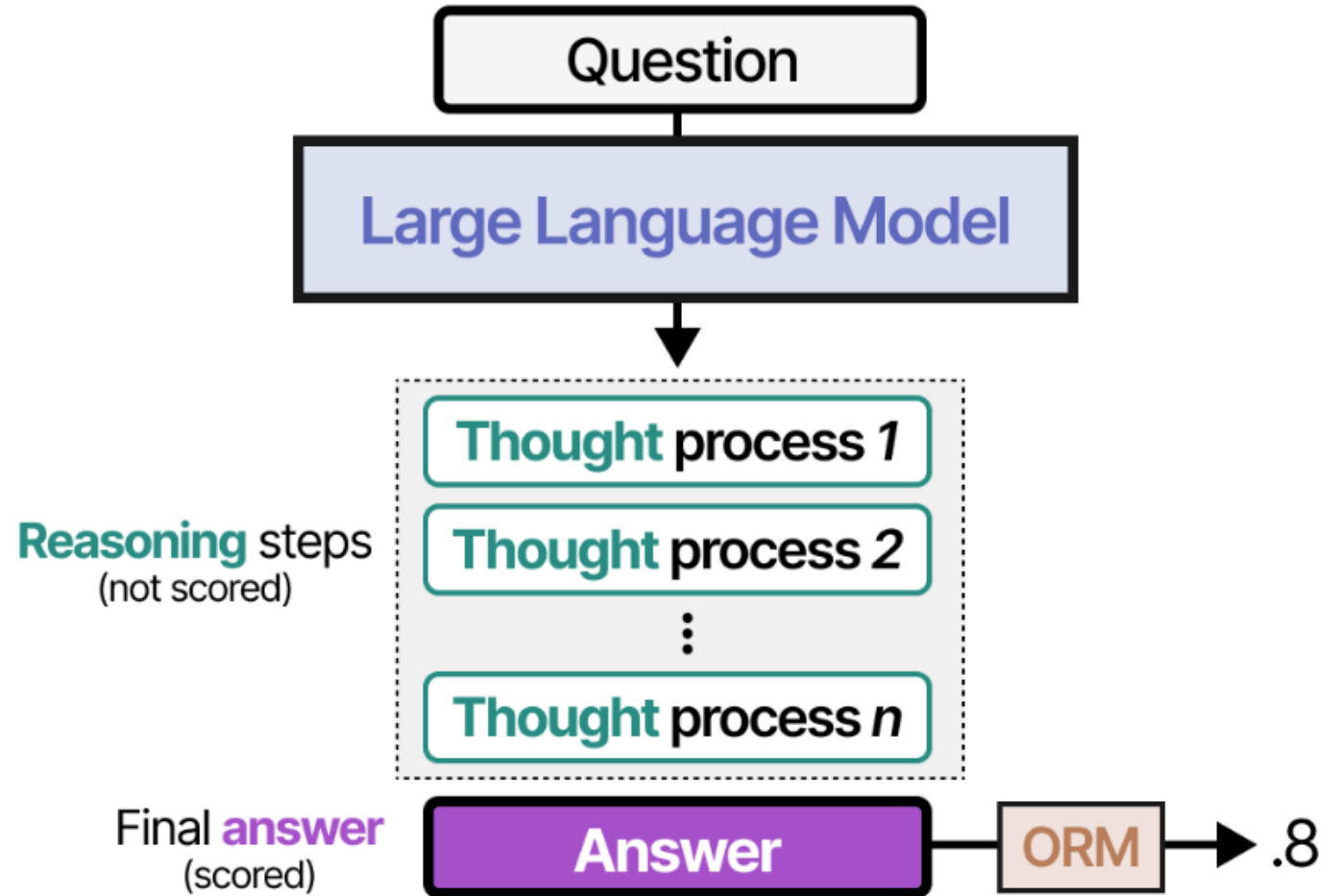
- **Reward models / verifiers** score generated answers and reasoning traces
- They may be **fine-tuned LLM judges** or **rule-based checks**
- **LLM-based verifiers**: assess quality, coherence, or usefulness of reasoning
- **Rule-based verifiers**: check objective properties such as correctness, format, or executable code

Two types of reward models

- **Outcome Reward Model (ORM)**: scores only the final answer
- Ignores how many reasoning steps were generated or how good those steps were
- **Process Reward Model (PRM)**: scores the intermediate reasoning steps
- Can reward sound progress even before the final answer is reached
- In practice, systems may combine **PRM + ORM** for both good reasoning and correct outcomes

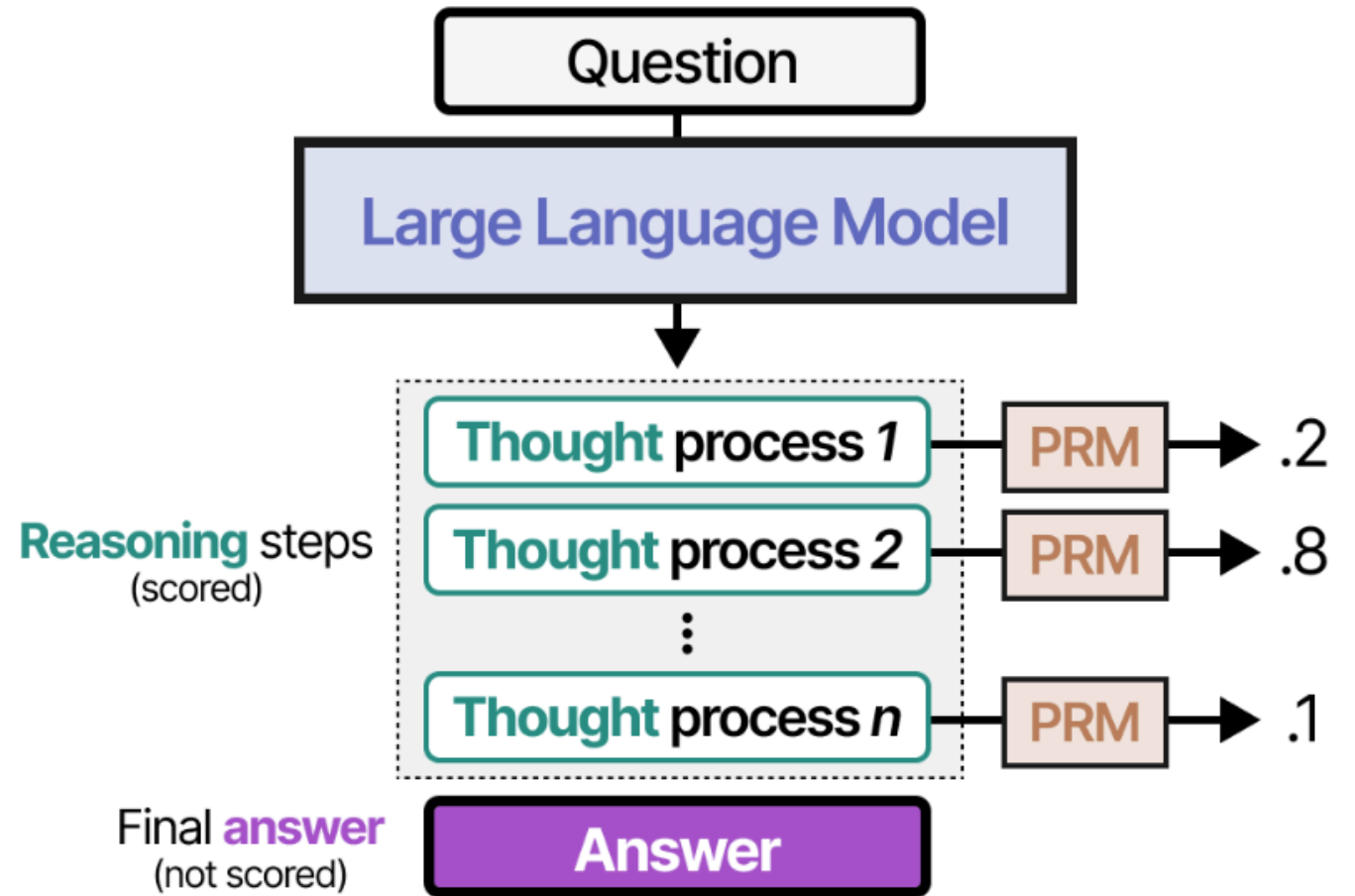
Reward Models: ORM

An output reward model (ORM) judges only the output and not intermediate reasoning steps



Reward Models: PRM

A process reward model (PRM) judges only the intermediate reasoning steps and not the output



Reward Models: PRM+ORM

Example

Question

I saw 6 flamingos. 2 flew away. 1 hid behind a tree. How many can I see?

Reasoning steps

First, you saw **6** flamingos. PRM → .9

1 flew away and you can see **3**. PRM → .1

I made a mistake. **2** flew away not **1**. So you can see **4**. PRM → .8

1 hid so now you can see **3**. PRM → .8

Final answer

3

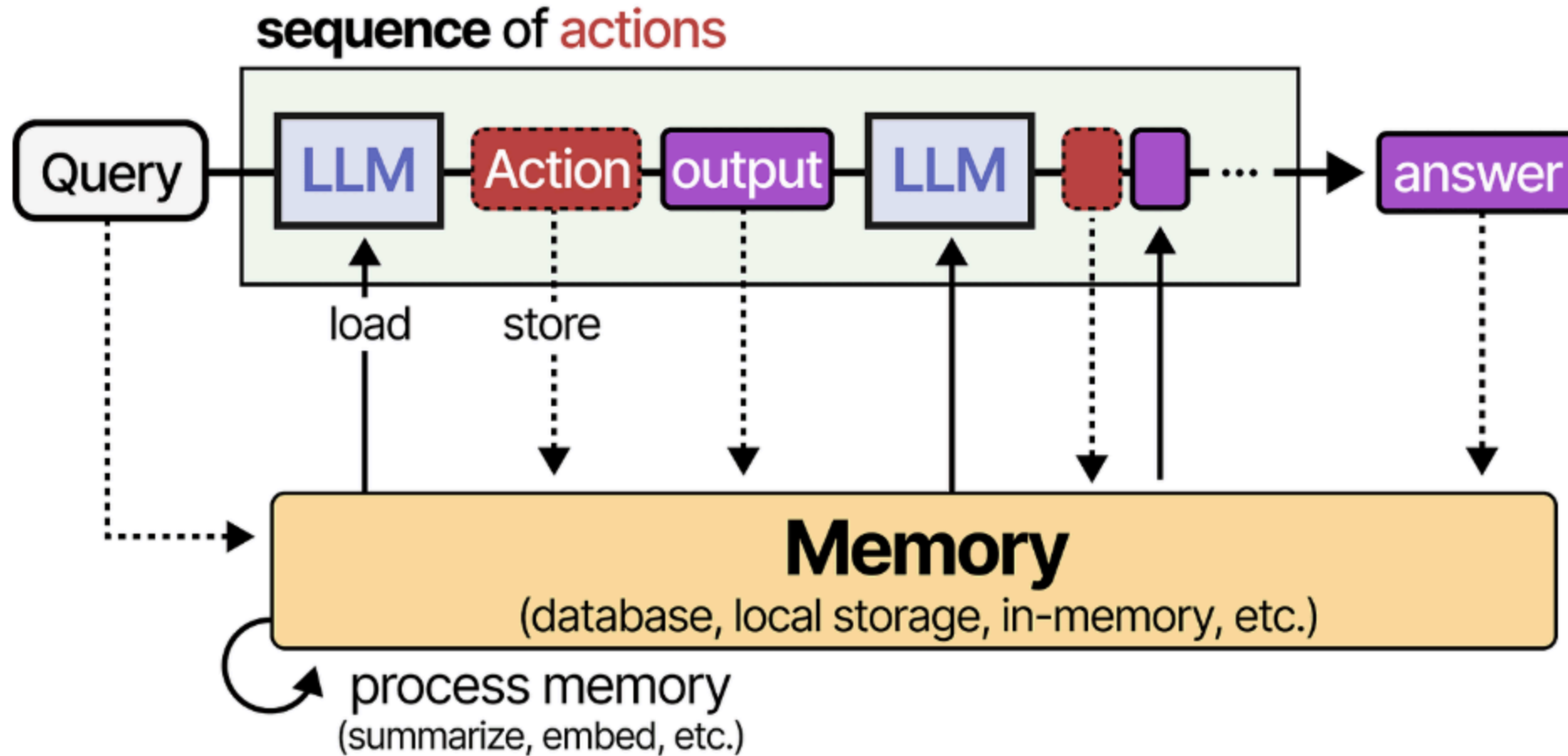
reasoning LLMs might also shown **reflective behavior**

LLMs are typically memoryless

Without memory



Adding Memory to LLMs



Short-term Memory

Short-term memory: they are generally formatted as messages demonstrating the differences between system prompts, the user's query, and the LLM's answer

messages =

```
[
  { role: system, content: "You are a helpful assistant." },
  { role: user, content: "Flamingos are my favorite animals." },
  { role: assistant, content: "That's wonderful!" },
  { role: user, content: "Tell me something about flamingos." },
  { role: assistant, content: "Flamingos are a type of ..." },
  |----- conversation history -----|
  { role: user, content: "What is my favorite animal?" },
  |----- query -----|
]
```

Short-term Memory

messages =

```
[  
  { role: system, content: "You are a helpful assistant." },  
  { role: user, content: "Flamingos are my favorite animals." },  
  { role: assistant, content: "That's wonderful!" },  
  { role: user, content: "Tell me something about flamingos." },  
  { role: assistant, content: "Flamingos are a type of ..." },  
  ..... conversation history .....  
  { role: user, content: "What is my favorite animal?" },  
  ..... query .....  
]
```

messages

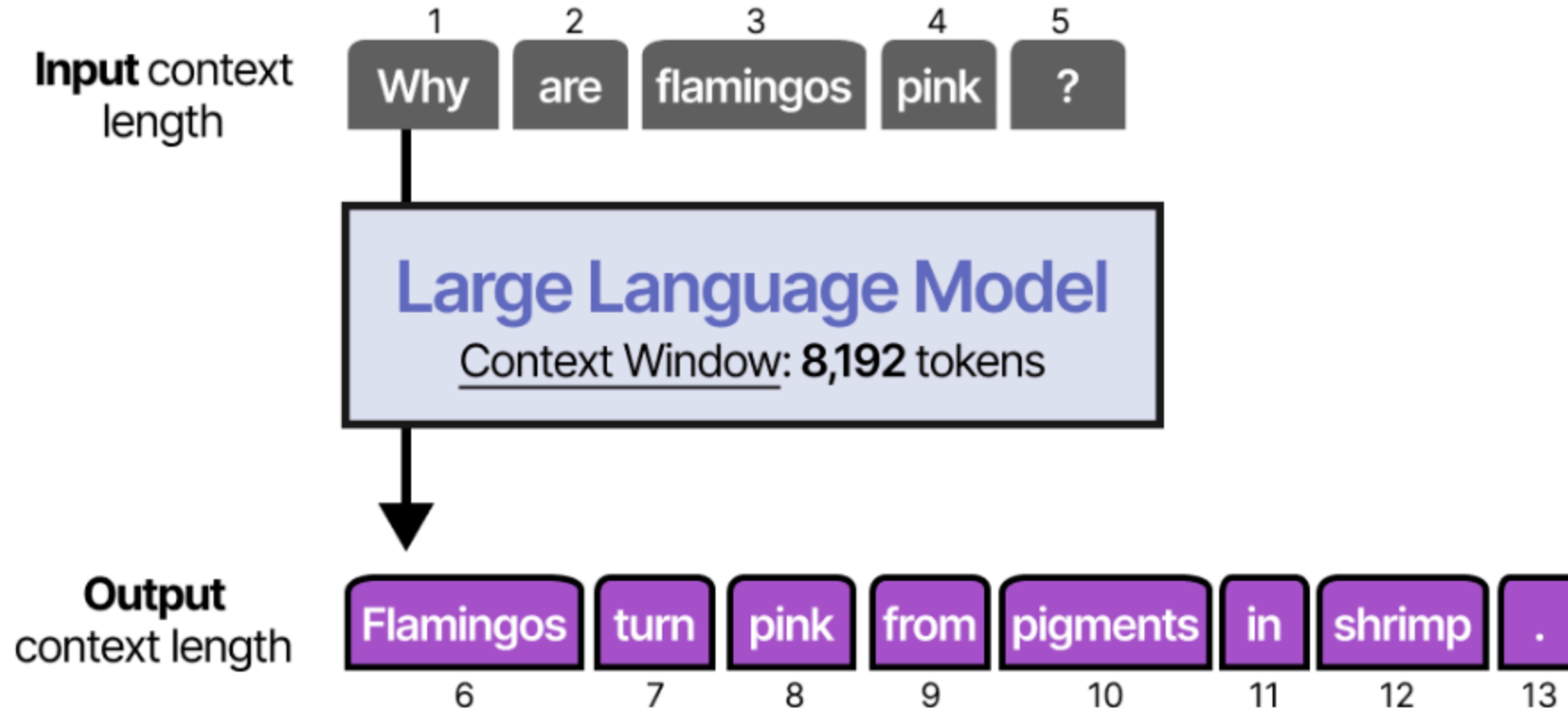
history + query

LLM

```
{  
  role: assistant,  
  content: "Flamingos!"  
}
```

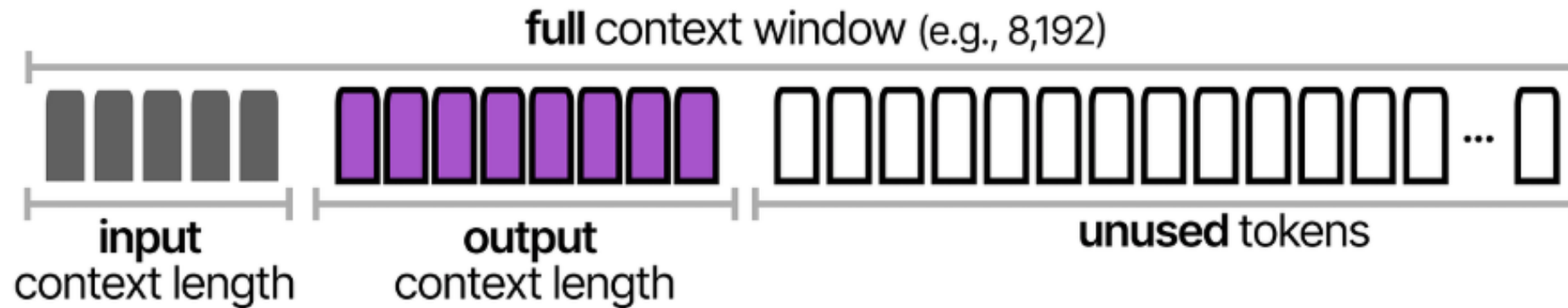
Short-term Memory: Context-Window

Most LLMs have a limited context window, which is the number of tokens that the LLM can process both and combine both the input and the output tokens



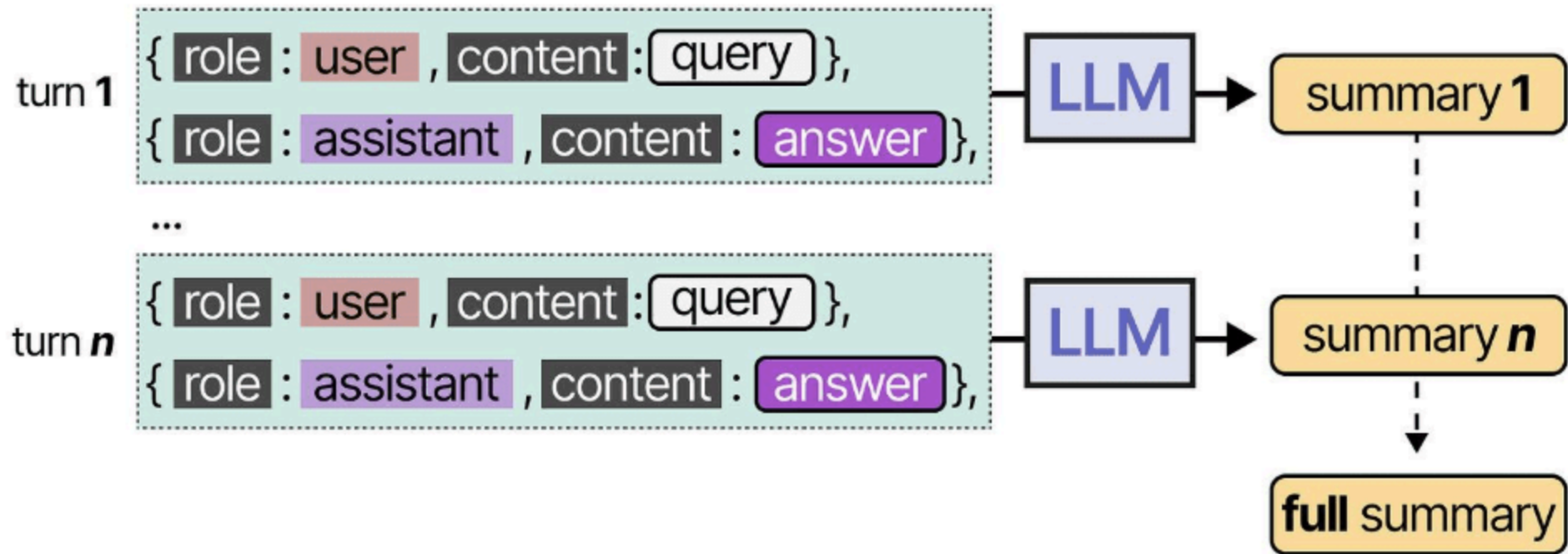
Short-term Memory: Context-Window

Most LLMs have a limited context window, which is the number of tokens that the LLM can process both and combine both the input and the output tokens



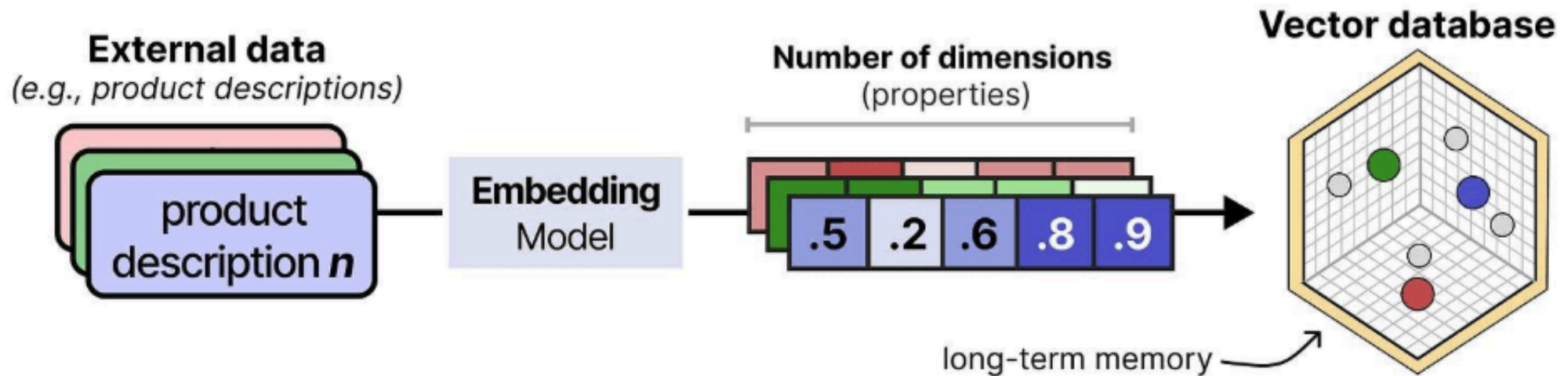
Short-term Memory: Context-Window

A common technique is to employ another LLM to summarize the conversation history. After each conversation turn, the same or another LLM will summarize it and add it to the full summary of the conversation



Long-term Memory

Long-term memory typically involves maintaining **one or more external databases** that can be queried to extract additional information.



- **Ingestion:** convert external documents into numerical embeddings
- **Embedding model:** maps similar meanings to nearby vectors
- **Vector database:** stores these embeddings as long-term external memory
- **Retrieval:** query the database to find relevant information later

Retrieval Augmented Generation (RAG)

- **Embed the user query** using the same embedding model
- **Retrieve relevant documents** by comparing query and document embeddings
- **Augment the prompt** with the retrieved context
- **Generate the answer** using the LLM and the added context
- Better retrieval usually leads to **more relevant and accurate responses**

